CLOUD INFRASTRUCTURE SPECIALIST

Architecting Landing Zones

Foundations to Production: Enterprise Multi-account Governance & Infrastructure as Code

OFFICIAL STUDENT TEXTBOOK

Level: Advanced | Duration: 14 Weeks

Hexadigitall Technologies

Copyright & Legal Notice

© 2026 Hexadigitall Technologies. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

Technical Disclaimer

The labs and architectures described in this textbook involve advanced cloud infrastructure configurations, including multi-account governance and hub-and-spoke networking. While every effort has been made to ensure the accuracy of the technical instructions, cloud providers frequently update their consoles and CLI tools.

Cloud Costs: Implementation of the architectures in this book, specifically AWS Organizations, Account Factory for Terraform (AFT), and Transit Gateways, may incur significant monthly costs. Students are responsible for monitoring their cloud spend and ensuring that resources are decommissioned after labs are completed.

Trademarks

Amazon Web Services (AWS) and the "Powered by AWS" logo are trademarks of Amazon.com, Inc. or its affiliates. Microsoft Azure and Azure Blueprints are trademarks of the Microsoft group of companies. Terraform is a trademark of HashiCorp. All other trademarks are the property of their respective owners.

Edition: Student Edition

Course Code: LZS-2026-ADV

Publisher: Hexadigitall Technologies

Official Portal: https://hexadigitall.com

Published in Nigeria, March 2026.

Preface: The Shift to Enterprise Scale

In the early stages of cloud adoption, a single-account setup is often sufficient. It provides a quick playground for innovation and a straightforward path to deployment. However, as organizations grow, this "flat" architecture quickly becomes a liability. The risks of a single-account setup—ranging from a massive blast radius during a security breach to hitting rigid API rate limits—create a glass ceiling for scalability.

The Landing Zone Specialist curriculum was designed to shatter that ceiling. This textbook is not just about learning cloud services; it is about mastering **Governance as Code**. We move beyond simple resource creation to focus on the skeletal system of the enterprise cloud: identity hierarchies, service control policies (SCPs), and hub-and-spoke networking architectures.

Why This Course?

The modern Cloud Engineer must be an architect who thinks in terms of "Account Factories" and automated guardrails. Over the next 14 weeks, you will transition from manual configurations to deploying standardized environments at scale using tools like Terraform and AWS Account Factory (AFT).

Our goal at **Hexadigitall Technologies** is to equip you with the mental models and hands-on experience required to lead a corporate handover. You aren't just building a cloud; you are building a secure, automated, and sustainable landing zone for the future of business.

"Scale is not an accident; it is a design choice."

The Hexadigitall Methodology: Engineering Your Career

This textbook is not just a collection of tutorials; it is a 14-week **Proof of Work** system. Every page is engineered using a tiered pedagogical approach based on Bloom's Taxonomy, ensuring you don't just "follow" instructions, but learn to **own** the architecture.

1. Foundations (Understand)

Pre-class exercises that prime your mental models. You start every week by setting up a dedicated folder in your GitHub repo for the upcoming architectural theme.

2. Teaching Notes (Analyze)

Deep-dives into the "Why." We analyze the trade-offs of SCP hierarchies and Hub-and-Spoke routing so you can defend your choices in a professional setting.

3. Hands-on Labs (Apply)

The "How." Deploying production-grade, multi-account infrastructure using Terraform and Cloud CLI. These labs result in clean, reusable code snippets.

4. Structured Projects (Create)

The "Proof." Independent builds designed for portfolio scaling. By the end of each week, you push an audited project folder to GitHub.

Accelerating Your Growth: Mentorship & Mastery

Architecting Landing Zones is a high-stakes endeavor. For those seeking to bridge the gap between "knowing" and "doing," we offer two distinct

pathways to professional mastery:

Pathway A: Modular Mentorship (Monthly)

Perfect for self-paced readers. Subscribe for the months you need support. Access a mentor to guide you through pre-class exercises, audit your GitHub pushes, and practice **Architectural Defense**—learning to justify your designs for high-level technical interviews. We monitor your progress and help you package your 14-week commit history as a world-class portfolio.

Explore Mentorship Options →

Pathway B: The Full Course Experience

Includes the textbook, full mentorship, and live cohort-based tutoring. This is for those who want a structured classroom environment, deep-dive peer reviews, and direct tutelage from the Hexadigitall engineering team.

Enroll in the Next Cohort →

Portfolio Tip: Recruiters value consistency. By the time you finish this textbook, your GitHub will show 14 distinct folders of enterprise-level work—creating a chronological audit trail of your expertise that outshines any traditional resume.

Need Additional Support?

📚 Resources & References →

⚠ Errata & Updates →

Entry Behavior & Prerequisites

This is a **Specialist Level** curriculum. To ensure success in architecting enterprise Landing Zones, students must move beyond basic resource deployment and possess a firm grasp of cloud-native infrastructure patterns.

Technical Baseline

- **Intermediate Terraform:** You should be comfortable with modules, state files, and variables. This course will advance those skills into multi-account state management and Account Factory automation.
- **Cloud Competency:** Experience with at least one major provider (AWS or Azure) at an Associate or Professional level. Specifically, familiarity with IAM, VPCs, and basic account-level security.
- **Git Workflow:** Proficiency in branching, committing, and pushing code. Your GitHub repository will serve as the primary audit trail for your mentorship and portfolio.

Environmental Readiness

Before beginning Week 1, your local workstation must be configured with the following:

```
- Terraform CLI (v1.5.0+)
- AWS CLI or Azure CLI
- Git (configured with SSH/GPG for signed commits)
- VS Code (Recommended extensions: HashiCorp Terraform)
```

Portfolio Architecture

We recommend creating a clean GitHub repository titled landing-zone-specialist-portfolio. Throughout this textbook, you will be instructed to create weekly folders (e.g., /week-01-foundations, /week-04-networking). This structure is essential for the monthly mentorship reviews and project defenses.

> **Account Warning:** You will require administrative access to a Master/Root account within a Cloud Organization. Be advised that certain Landing Zone services (e.g., Transit Gateway, Control Tower) are not covered by the "Free Tier."

Detailed Table of Contents

COURSE PREPARATION & MENTORSHIP GUIDE

Course Preparation: Mentor onboarding, portfolio structure, and how to use this textbook.

Includes: mentor review cadence, portfolio repository structure, weekly artifact rules, and professional delivery expectations.

PHASE I: FOUNDATIONAL STRATEGY & MULTI-ACCOUNT THEORY

Week 1: Landing Zone Strategy & Blast Radius Control

Includes: architectural decision log, blast radius analysis, governance-ready account segmentation and portfolio baseline.

Week 2: Identity & Management Plane Architecture

Includes: organizational hierarchy, identity source-of-truth, OU design, and role inheritance mapping.

Week 3: Governance, Guardrails & Secure Access Controls

Includes: policy guardrails, SCP/permission boundary strategy, SSO integration, and cost-governance alignment.

PHASE II: NETWORK ARCHITECTURE & SHARED SERVICES

Week 4: Enterprise Network Architectures & Transit Control

Includes: hub-and-spoke topology, transit gateway routing, segmentation patterns, and hybrid readiness.

Week 5: Shared Services, Private Link & Secure Connectivity

Includes: secure shared service design, private connectivity patterns, service catalog integration, and edge trust boundaries.

Week 6: Automated Security & Detection Engineering

Includes: centralized security telemetry, guard duty automation, detection engineering, and response readiness.

Week 7: Observability, Logging & Audit Pipeline

Includes: telemetry architecture, log centralization, audit collection, and monitoring dashboard design.

PHASE III: AUTOMATION, DELIVERY & PLATFORM GOVERNANCE

Week 8: Automated Account Factory & Self-Service Provisioning

Includes: account vending automation, secure IaC pattern, environment promotion, and state governance.

Week 9: Account Customizations, Post-Deployment Controls & Drift Guardrails

Includes: custom account policies, post-deployment lifecycle controls, drift detection, and remediation workflow.

Week 10: Platform Delivery with AFT & Terraform Automation

Includes: delivery pipelines, change management, gated releases, and deployment safety.

Week 11: Standardization, Blueprints & Service Catalog Onboarding

Includes: platform standardization, blueprint governance, workload onboarding playbooks, and service catalog patterns.

PHASE IV: LIFECYCLE, SCALE & CAPSTONE DELIVERY

Week 12: Compliance Validation, Audit Evidence & Lifecycle Guardrails

Includes: compliance evidence, control mapping, audit readiness, and operational lifecycle guardrails.

Week 13: State Management, Scale Strategy & Resilience Planning

Includes: state management at scale, environment drift strategy, recovery planning, and resilience readiness.

Week 14: Capstone Delivery, Executive Handover & Architecture Defense

Includes: final architecture dossier, executive handover package, portfolio presentation, and defense preparation.

COURSE PREP

Course Preparation & Mentorship Guide

This week establishes the mentor-led structure, the portfolio repository rules, and the weekly artifact workflow that will make every later chapter a professionally packaged architecture delivery.

"A professional architecture engagement begins with the process just as much as the design. This week, we build the process."

PREPARATION

Course Preparation & Mentorship Guide

WEEKLY TECHNICAL SCOPE

1. **Orientation:** Mentor review cadence, artifact expectations, and weekly check-in structure.
2. **Setup:** Portfolio repo structure, weekly folder naming, and Git commit hygiene.
3. **Deliverables:** README, course roadmap, and mentor-ready presentation plan.
4. **Checkpoint:** First mentor review, feedback loop, and defense preparation plan.

The value of this course is not only in the technical build; it is also in the habit of delivering polished, reviewable artifacts every week. This week sets the standard.

"A well-structured portfolio is the best proof that you can do the work again, and better."

Orientation & Mentorship Workflow

This textbook is organized as a mentor-guided delivery cycle. Each week you will produce reviewable artifacts, submit them to your mentor, and use feedback to improve the next week's architectural work.

How to Use This Textbook

Start each chapter by reading the outcome summary and definition of the weekly deliverable. Then complete the repository action, document your reasoning, and update your portfolio folder. Treat the chapter as a professional engagement, not a lab assignment.

Weekly Mentor Review Cycle

Each week follows the same rhythm: prepare the artifact, submit it into the portfolio repo, share it with your mentor, receive feedback, and revise. This cadence is designed to build discipline and make your progress visible.

1. Prepare: Complete the weekly analysis, architecture artifact, and supporting documentation.

2. Submit: Push the work to the portfolio repo and notify your mentor with a clear summary of what changed.

3. Review: Mentor evaluates the work, scores it, and provides constructive guidance.

4. Revise: Implement feedback, refine the narrative, and document the decision rationale in the next weekly folder.

Portfolio Repository Rules

- Repo Name: Use landing-zone-specialist-portfolio as the canonical repository for this course.

- Folder Structure: Create weekly folders such as /week-01-foundations, /week-02-identity, and inside each include analysis.md, architecture.md, and supporting artifacts.

- Deliverables: Each week should include a README or summary document, a design artifact, and evidence of the implementation decisions.

- Git Hygiene: Use consistent commit messages, one logical change per commit, and preserve a clean history that demonstrates progression.

Deliverable Checklist Per Week

- Analysis: A problem statement, success criteria, and risk summary.

- Architecture: Diagrams, account design, and policy decisions.

- Code or Configuration: Terraform, CLI scripts, or architecture-as-code assets.

- Review Notes: Mentor questions, feedback status, and the next step plan.

- Executive Summary: A concise portfolio statement that explains what was built and why it matters.

How Your Mentor Scores Each Week

Mentor scoring is based on the professionalism, clarity, and completeness of your weekly delivery. These are the core dimensions:

- Clarity: Can a reviewer quickly understand the problem, approach, and result?

- **Accuracy:** Are the architectural concepts and implementation details correct?

- **Completeness:** Are the expected artifacts present and organized?

- **Narrative:** Does the work tell a coherent story that fits the enterprise use case?

- **Professionalism:** Is the deliverable polished, documented, and ready for a decision-maker?

How This Course Prepares You for Interviews & Enterprise Proposals

Every weekly portfolio artifact is also a story you can use in architecture interviews and proposal work. This course teaches you to document tradeoffs, justify decisions, and present a coherent enterprise solution—not just to build infrastructure.

Orientation Deliverables

- **Portfolio Repository:** A GitHub repo named landing-zone-specialist-portfolio with weekly folders and a polished README.

- **Weekly Review Cycle:** A clear mentor submission process for feedback, review, and revision.

- **Process Guide:** A written expectation summary for mentor handoff and score criteria.

- **Career Narrative:** An introduction that explains how this portfolio demonstrates enterprise architecture readiness.

This section is intentionally about the way we work, not the minute details of your CLI environment. The technical readiness checklist remains in the front matter, while this guide explains how to turn the course into a mentor-reviewed portfolio.

> ***Mentor Note:*** *This guide is your operating model. Your mentor will score the week on the quality of the outcome, the structure of the artifact, and your ability to translate technical work into enterprise value.*

PHASE I

Foundations & Multi-Account Theory

An exploration into the architectural pivot from single-account environments to scalable, governed enterprise cloud structures. We establish the "Why" before we build the "How."

"Architecture is the art of how to waste space beautifully; in the cloud, it is the art of how to manage scale securely."

WEEK 01

Landing Zone Strategy & Blast Radius Control

WEEKLY TECHNICAL SCOPE

1. **Foundation:** Entry Behavior & Blast Radius Theory.
2. **Teaching Notes:** Cloud Provider Service Quotas & API Rate Limits.
3. **Hands-on Lab:** Organization Health Audit via CLI.
4. **Weekly Project:** Portfolio Initialization & Baseline Documentation.

Before deploying a single resource, we must understand the constraints of the cloud. This week focuses on the physical and logical limits that necessitate a multi-account strategy, while also establishing the mentor-ready portfolio, weekly documentation rhythm, and baseline deliverables your mentor will review.

"The first step in building a resilient system is acknowledging that everything fails eventually. The second step is making sure it doesn't matter when it does."

Chapter 1: The Blast Radius Theory

In a traditional single-account cloud environment, every resource shares the same logical boundary. This shared boundary is known as the **Blast Radius**. If credentials are compromised, the entire organization is at risk. Architecting a Landing Zone is primarily an exercise in **Blast Radius Reduction**.

The Case for Account-Level Segregation

Architecting a Landing Zone is primarily an exercise in **Blast Radius Reduction**. By moving to a multi-account strategy, we ensure that:

01. **Security Isolation:** A compromise in a "Sandbox" account cannot move laterally into the "Production" account.
02. **Quotas & Rate Limits:** Heavy workloads in one account won't cause "Throttling" errors for critical services in another.
03. **Billing Transparency:** Costs are naturally aggregated by account, making chargebacks to different departments seamless.

> ***Architect's Note:*** *When we build the Organization hierarchy in Week 2, our primary goal will be to design OUs (Organizational Units) that mirror these blast radius requirements.*

Pre-Class Exercises: Initializing the Proof of Work

To support the 14-week "Proof of Work" cycle, you must initialize your local and remote environments. Complete these exercises before moving to the Teaching Notes:

Ex 1. **Repository Creation:** Create a new GitHub repository titled landing-zone-specialist-portfolio. Initialize it with a README.md and a .gitignore specifically for Terraform.

Ex 2. **Directory Versioning:** Create your first structural folder: /week-01-foundations. Inside, create a subfolder named /exercises.

Ex 3. **CLI Connectivity:** Authenticate your Cloud CLI (AWS/Azure) and run a command to list your current account ID. Redirect this output to a file named account_info.txt inside your week 1 exercises folder.

> ***Mentorship Note:*** *If you are enrolled in the* ***Modular Mentorship****, your mentor will audit these first commits to ensure your Git signature and folder structure meet enterprise standards.*

The Glass Ceiling: Service Quotas & API Throttling

Security is the most common reason for a multi-account strategy, but **operational physics** is often the most urgent. Every cloud account has invisible "hard" and "soft" limits that, once hit, can bring an entire enterprise to a standstill.

1. Service Quotas (Resource Limits)

Cloud providers limit the number of resources (VPCs, EC2 instances, IGWs) you can create per region per account. While some are adjustable, they represent a shared pool. In a single-account setup, a developer in "Dev" could accidentally consume all available Elastic IPs, preventing "Production" from scaling during a traffic spike.

2. API Rate Limits (The Throttling Problem)

Every time you run a terraform plan or a script calls DescribeInstances, you consume API credits. Cloud providers throttle these requests to protect their control plane.

> **Scenario:** If you have 50 microservices in one account all being managed by a CI/CD pipeline, the sheer volume of API calls can trigger RequestLimitExceeded errors. This "noisy neighbor" effect can block critical security patches from deploying.

The Landing Zone Solution

By distributing workloads across multiple accounts, we effectively **multiply our API quota** and resource limits by the number of accounts in the Organization. This allows the enterprise to grow horizontally without ever hitting the "Glass Ceiling" of a single-account's constraints.

Lab 1.1: Auditing Organization Health

In this lab, you will use the Command Line Interface (CLI) to identify the current limits of your account. This data provides the baseline evidence needed to architect a multi-account structure.

Task 1: Identifying Account Constraints

Execute the following command to view your current service quotas for VPCs. This identifies how many "spoke" networks you can create before hitting a hard limit.

```
# AWS Example: List VPC Quotas
aws service-quotas get-service-quota \
  --service-code vpc \
  --quota-code L-F678F1CE
```

Task 2: Detecting "Noisy Neighbors"

Review your CloudTrail logs to identify which IAM entities are making the most frequent API calls. This exercise simulates identifying an application that might trigger API throttling for the whole account.

Lab Requirements

- [ ] Capture a screenshot of your VPC quota output.
- [ ] Identify one "Adjustable" quota vs one "Non-adjustable" quota.
- [ ] Save these findings into your /week-01-foundations/exercises folder.

> **Architectural Defense:** Why would an enterprise prefer to open a second account rather than simply requesting a quota increase for an existing one? Prepare to answer this in your project defense.

Project 1: Portfolio Initialization & Baseline Documentation

The objective of this project is to establish the **Infrastructure as Code (IaC)** standards for your 14-week portfolio and document the "Pre-Landing Zone" state of your environment.

Project Deliverables

1. Standardized Repository Structure

Finalize your GitHub structure. Your repository must now include a top-level docs/ folder and a projects/ folder. Week 1 findings should be migrated into projects/week-01-foundations/.

2. The "Blast Radius" Audit Report

Create a Markdown file titled ENVIRONMENT_AUDIT.md. Using the data from Lab 1.1, document at least three service quotas that would act as blockers for an enterprise scaling to 50+ VPCs in a single account.

3. Portfolio README Enhancement

Update your main README.md to reflect the "Landing Zone Specialist" mission. Include a "Tech Stack" section listing Terraform, Cloud CLI, and Git.

Submission Checklist

Signed Commits	Are all commits GPG or SSH signed?
Folder Hierarchy	Is the naming convention consistent (kebab-case)?
Documentation	Does the Audit Report include raw CLI output snippets?

Career Strategy: This project is your first "Commit." It demonstrates to future employers that you possess the administrative discipline required to audit existing infrastructure before attempting to migrate it.

WEEK 02

Identity & Management Plane Architecture

WEEKLY TECHNICAL SCOPE

1. Foundation: Hierarchical Management & OU Design Patterns.

2. Teaching Notes: AWS Organizations vs. Azure Management Groups.

3. Hands-on Lab: Provisioning Multi-Account Structures with Terraform.

4. Weekly Project: Building the "Golden" Organizational Hierarchy.

Having established the necessity of multiple accounts, we now focus on the management plane. This week introduces the skeletal structure of the enterprise cloud: Organizational Units (OUs). We will learn to design hierarchies that balance security isolation with operational ease, setting the stage for global governance.

"Structure is not a constraint on creativity; it is the foundation that allows creativity to scale without collapsing into chaos."

Chapter 2: Hierarchical Management Patterns

Once an enterprise moves beyond a single account, the primary challenge shifts from resource management to **governance management**. We solve this by implementing a hierarchical structure using Organizational Units (OUs). An OU is a logical container for accounts that allows you to apply policies to multiple entities simultaneously, ensuring uniform security standards.

The Lifecycle-Based vs. Functional Hierarchy

There are two primary patterns for designing your Landing Zone hierarchy:

P1. **Lifecycle Pattern:** Organizing OUs by environment (e.g., Development, Staging, Production). This is ideal for applying strict security guardrails that only trigger in production.

P2. **Functional Pattern:** Organizing OUs by business unit or project (e.g., Finance, Engineering, Marketing). This is preferred for complex organizations with decentralized IT budgets.

Pre-Class Exercises: Scaling the Portfolio

To prepare for the automated deployment of your Organization, complete these structural tasks:

Ex 1. **Portfolio Expansion:** Create a new directory /week-02-identity in your repository. Inside, create subfolders for /exercises and /lab-code.

Ex 2. **Documentation:** In a file titled HIERARCHY_DESIGN.md, sketch a 3-level hierarchy for a fictional company with "Security," "Core," and "Workloads" OUs.

Ex 3. **State Prep:** Initialize a new Terraform configuration file in your /lab-code folder. Configure the provider to target your cloud organization's management account.

Architect's Note: *The choices you make in your hierarchy design this week will dictate where you apply Service Control Policies (SCPs) in Week 3. Design for long-term stability.*

The Management Plane: AWS vs. Azure

To build a truly cross-platform Landing Zone, an architect must understand how different providers handle account grouping. While AWS uses **Organizations**, Azure utilizes **Management Groups**. Both structures allow for policy inheritance, but their implementation logic varies.

1. AWS Organizations (OUs)

In AWS, the **Management Account** sits at the top. It creates **Organizational Units (OUs)**, which are containers for accounts. A key constraint is that an AWS account can only belong to exactly one OU at a time. Policies (SCPs) applied to an OU are inherited by all child OUs and accounts beneath it.

2. Azure Management Groups

Azure takes a slightly more flexible approach. **Management Groups** sit above **Subscriptions** (Azure's equivalent of an AWS account). Unlike AWS, Azure allows for a deep nesting of up to six levels of Management Groups, providing a more granular control over complex corporate hierarchies.

Feature	AWS	Azure
Root Container	Organization Root	Root Management Group

Grouping Unit	Organizational Unit (OU)	Management Group
Individual Unit	Account	Subscription

> ***Key Strategy:*** *Regardless of the provider, always avoid placing accounts or subscriptions directly in the "Root." Always create at least one layer of management containers to prevent accidental global policy lockouts.*

Lab 2.1: Provisioning Multi-Account Structures

This lab focuses on automating the creation of Organizational Units (OUs) using Terraform. By the end of this session, you will have a live, version-controlled hierarchy within your Cloud Organization.

Task 1: Define the Root and Child OUs

Create a main.tf file in your /week-02-identity/lab-code directory. Use the snippet below as a template to define your first two management layers.

```
resource "aws_organizations_organizational_unit" "security" {
  name      = "Security"
  parent_id = data.aws_organizations_organization.org.roots[0].id
}

resource "aws_organizations_organizational_unit" "workloads" {
  name      = "Workloads"
  parent_id = data.aws_organizations_organization.org.roots[0].id
}
```

Task 2: Deployment and Validation

Run the following Terraform commands. Ensure your CLI is authenticated with the **Management Account** of your Organization.

- [ ] terraform init - Initialize the backend and providers.
- [ ] terraform apply - Execute the plan to create the OUs.
- [ ] Verify the new structure in the Cloud Console or via CLI: aws organizations list-roots.

> **Architectural Defense:** If you delete the Terraform state file, what happens to the OUs in your cloud account? Understanding the difference between *Resource Management* and *State Management* is key for this week's project.

Project 2: Building the "Golden" Organizational Hierarchy

The objective of this project is to deploy a multi-layered Organizational Unit (OU) structure that separates core infrastructure from business workloads. This "Golden" hierarchy is the industry standard for minimizing risk across the enterprise.

Project Deliverables

1. The Nested OU Configuration

Extend your Terraform code to include nested OUs. You must create a "Workloads" parent OU with "Development" and "Production" as child OUs. Ensure all resource dependencies are correctly mapped in your IaC.

2. Hierarchy Visualization Map

Create a file named ORGANIZATION_MAP.md in your /week-02-identity folder. Provide a visual tree (using ASCII or a diagram tool) that maps out your final root, OUs, and child OUs.

3. Scalability Strategy Write-up

Document why you chose a Lifecycle-based vs. Functional-based hierarchy. Explain how this structure prevents "Production" resources from being affected by "Development" mistakes.

Submission Checklist

IaC Versioning	Is your Terraform code pushed to the /lab-code folder?
Resource Integrity	Do your OUs follow a consistent naming convention?
Commit History	Does your GitHub history show the evolution from Week 1 to Week 2?

Career Strategy: In professional interviews, you will be asked: "How do you organize 500 accounts?" Pointing to this project demonstrates that you don't just know the tools—you know the **architecture of scale**.

WEEK 03

Governance, Guardrails & Secure Access Controls

WEEKLY TECHNICAL SCOPE

1. **Foundation:** Least Privilege & Global Guardrail Theory.
2. **Teaching Notes:** SCPs vs. Permission Boundaries vs. IAM.
3. **Hands-on Lab:** Deploying Global Guardrails as Code.
4. **Weekly Project:** Automating Identity Center (SSO) Provisioning.

Advanced Topic: FinOps & Cost Tagging Architecture
Build a reusable Terraform module for cost allocation tagging that enforces tag compliance across all accounts. Create a cost chargeback runbook documenting how billing is attributed to OUs and team owners, plus 3 cost optimization recommendations based on your landing zone design.

In Week 2, we built the house; this week, we install the security system. Governance in a Landing Zone is about "Maximum Freedom within a Defined Perimeter." We will explore how to use Service Control Policies (SCPs) to set absolute limits that even an Administrator cannot bypass, ensuring your "Production" accounts remain immutable and secure.

"Good governance is not about saying 'No'; it is about building a 'Yes' that is safe, scalable, and impossible to break."

Chapter 3: Least Privilege & Guardrail Theory

In a multi-account environment, the **Principle of Least Privilege (PoLP)** is no longer just about individual users; it is about the accounts themselves. A **Guardrail** is a high-level governance policy that provides ongoing oversight of your environment. Unlike traditional firewalls, guardrails are preventive or detective measures that stay in effect even as your infrastructure evolves.

Preventive vs. Detective Guardrails

A specialist architect must balance the rigidity of prevention with the agility of detection:

G1. **Preventive Guardrails:** These stop an action before it happens (e.g., an SCP that prevents any user in a "Production" OU from deleting CloudTrail logs).

G2. **Detective Guardrails:** These identify when a resource is "Out of Compliance" and trigger an alert or automated remediation (e.g., a rule that flags any S3 bucket that is made public).

Pre-Class Exercises: Preparing for Governance

To prepare for deploying global policies, initialize your Week 3 workspace:

Ex 1. **Directory Setup:** Create /week-03-governance. Inside, create /policies and /sso-config.

Ex 2. **Policy JSON Drafting:** In your /policies folder, create a file named deny-region-lock.json. Write a draft policy that would deny all actions in every region except your primary home region (e.g., us-east-1 or eu-west-1).

Ex 3. **Identity Audit:** List the current IAM users in your management account and identify which ones should be migrated to AWS IAM Identity Center (SSO).

Architect's Note: *Service Control Policies (SCPs) do not grant permissions; they only filter them. Even if a user has "AdministratorAccess," an SCP can still block them. This is the ultimate "Root" of cloud security.*

The Policy Intersection: SCPs vs. Boundaries vs. IAM

In a Landing Zone, permissions are not just about what a user *has*; they are about what the environment *permits*. We use a "defense-in-depth" approach where three distinct policy types intersect to determine the final effective permission.

1. Service Control Policies (SCPs)

SCPs are the "Global Filters." They are applied at the OU or Account level within the Organization. An SCP defines the **maximum available permissions** for an entire account. Even if a user is given "Full Administrator" rights via IAM, if an SCP denies a service (e.g., "Deny S3"), that user can never access S3.

2. Permission Boundaries

These are the "Safety Nets" for decentralized administration. We use Permission Boundaries to limit the maximum permissions an IAM entity can have. This is essential when you want to allow a developer to create their own IAM roles but want to ensure they cannot create a role with more power than themselves.

3. Identity-Based (IAM) Policies

These are the "Standard Access" grants. They are attached directly to users, groups, or roles. They define exactly what actions the identity can perform on specific resources.

The Evaluation Logic Rule:

Implicit Deny → SCP Check → Boundary Check → IAM Check → Final Decision.

If *any* layer contains an explicit "Deny," the request is rejected immediately, regardless of what other layers say.

Lab 3.1: Deploying Global Guardrails as Code

This session focuses on codifying a "Region Lock" policy. This is a standard enterprise guardrail used to ensure data residency compliance and to prevent cost spikes from resources deployed in non-standard regions.

Task 1: Define the Region Lock Policy

Create a guardrails.tf file in your /week-03-governance/policies directory. This code defines the policy and attaches it to your "Workloads" OU.

```
resource "aws_organizations_policy" "region_lock" {
  name        = "RegionLockPolicy"
  description = "Deny all regions except us-east-1"
  content     = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Sid      = "DenyAllOutsideUSEast1"
      Effect   = "Deny"
      NotAction = ["iam:*", "organizations:*", "route53:*"]
      Resource = "*"
      Condition = {
        StringNotEquals = { "aws:RequestedRegion": ["us-east-1"] }
      }
    }]
  })
}

resource "aws_organizations_policy_attachment" "workloads_attach" {
  policy_id = aws_organizations_policy.region_lock.id
  target_id = aws_organizations_organizational_unit.workloads.id
}
```

Task 2: Testing and Denial Verification

After running terraform apply, switch to a child account within the Workloads OU and attempt to create a resource in a forbidden region (e.g., us-west-2).

- [ ] Run terraform apply to push the SCP to the Cloud Organization.
- [ ] Attempt to create a VPC in us-west-2 and confirm the "Explicit Deny" error.
- [ ] Document the error message in your ENVIRONMENT_AUDIT.md for this week.

> **Architectural Defense:** Why did we include iam:* and route53:* in the NotAction list? Hint: Think about "Global Services" that do not belong to a specific region.

Project 3: Automating Identity Center (SSO) Provisioning

The objective of this project is to eliminate "Long-Lived Credentials" by deploying a centralized Identity Center. You will codify Permission Sets and Assignments to allow seamless access across your multi-account hierarchy.

Project Deliverables

1. The "Specialist" Permission Set

Create a Terraform resource for an SSO Permission Set titled LandingZoneSpecialist. It must include a session duration of 4 hours and be attached to the ReadOnlyAccess managed policy.

2. Automated Account Assignments

Codify the assignment of your "Specialist" user to both the Security and Workloads OUs. Use Terraform for_each loops to ensure that as new accounts are added to these OUs, permissions are inherited automatically.

3. The Identity Migration Report

Document the transition in IDENTITY_STRATEGY.md. Explain why moving to SSO reduces the risk of credential leakage compared to static IAM Access Keys.

Phase I Completion Checklist

Governance Proof	Is your Region-Lock SCP active and tested?
Identity Proof	Can you log in via the SSO portal without using an IAM password?
Repository Health	Does your GitHub repository show 14+ days of consistent activity?

Career Strategy: Transitioning an organization from IAM Users to Identity Center is one of the most common "Quick Wins" a consultant can provide. This project is your template for that high-value service.

Phase I Knowledge Assessment

Assessment Overview

Questions: 15 multiple-choice questions
Time Estimate: 45-60 minutes
Passing Score: 11/15 (73%)
Coverage: Weeks 1-3 (Account Strategy, Identity, Governance)

1. What is the primary purpose of the "Blast Radius" concept in Landing Zone design?

A) To minimize cloud costs by reducing resource usage
B) To limit the impact of security breaches or configuration errors
C) To optimize network performance across regions
D) To standardize resource naming conventions

2. Which AWS service provides the foundational structure for multi-account governance and centralized billing?

A) AWS Control Tower
B) AWS Organizations
C) AWS Resource Access Manager
D) AWS Systems Manager

3. In AWS Organizations, what is the hierarchical structure used to group accounts with similar purposes?

A) Resource Groups
B) Organizational Units (OUs)
C) Security Groups
D) Auto Scaling Groups

4. What is the primary function of AWS IAM Identity Center (formerly SSO)?

A) To manage EC2 instances across accounts
B) To provide centralized user authentication and authorization
C) To monitor resource usage and costs
D) To automate infrastructure deployment

5. Which policy type in AWS Organizations acts as a permissions boundary that cannot be overridden by IAM policies?

A) Identity-based policies
B) Resource-based policies
C) Service Control Policies (SCPs)
D) Trust policies

6. What is the recommended approach for managing IAM users in a Landing Zone?

A) Create individual IAM users in each account
B) Use the root account for all administrative tasks
C) Centralize identity management and avoid long-term IAM credentials
D) Share access keys between team members

7. Which account type should contain the foundational AWS services and organizational policies?

A) Workload accounts
B) Management account
C) Log Archive account
D) Audit account

8. What is the purpose of a "Permission Set" in AWS IAM Identity Center?

A) To define user passwords
B) To specify the level of access users have in assigned accounts
C) To manage MFA settings
D) To track user login history

9. Which principle should guide the design of Organizational Units in a Landing Zone?

A) Minimize the number of OUs for simplicity
B) Group accounts by business function and risk profile
C) Create one OU per AWS region
D) Base OU structure on team size

10. What is the effect of attaching an SCP with a "Deny" statement to an OU?

A) It grants permissions to all accounts in the OU
B) It overrides any IAM Allow policies for the denied actions
C) It only affects the root user of the Management account
D) It requires approval from the account owner to take effect

11. Which account should be used for day-to-day administrative tasks in a Landing Zone?

A) Workload accounts
B) A dedicated administrative workload account
C) The Log Archive account
D) Any account with administrative permissions

12. What is the primary benefit of using AWS Organizations for cost management?

A) It automatically reduces costs
B) It provides consolidated billing and cost allocation tags
C) It replaces individual account billing
D) It manages resource reservations

13. Which AWS service helps prevent the creation of resources in unauthorized regions?

A) AWS Config
B) Service Control Policies
C) AWS Budgets

D) AWS Trusted Advisor

14. What should be the first step when designing a Landing Zone OU structure?

A) Create as many OUs as possible
B) Understand business requirements and compliance needs
C) Copy an existing organization's structure
D) Focus only on technical requirements

15. Which principle is violated by using long-term IAM access keys in a Landing Zone?

A) Least Privilege
B) Defense in Depth
C) Zero Trust
D) All of the above

🎯 **Phase I Completion**

Congratulations on completing Phase I! You now understand the foundational principles of multi-account governance.

Proceed to Phase II to learn about network architecture and shared services. Your mentor will review your Phase I work before advancing.

Phase I Assessment

PHASE II

Networking & Shared Services

Moving from identity to connectivity. This phase covers the construction of the "Data Highway" of your Landing Zone—designing Hub-and-Spoke architectures, centralizing security inspections, and automating cross-account logging.

"In the cloud, networking is no longer about cables and switches; it is a software-defined conversation between distributed entities."

WEEK 04

Enterprise Network Architectures & Transit Control

WEEKLY TECHNICAL SCOPE

1. Foundation: Inter-account Communication Theory & Transit Gateway Logic.

2. Teaching Notes: Transit Gateway vs. VPC Peering at Scale.

3. Hands-on Lab: Automated Network Hub Deployment using Terraform.

4. Weekly Project: Building a Centralized Logging & Audit Repository.

In Phase I, we secured the identity of our accounts. In Phase II, we connect them. This week introduces the "Hub-and-Spoke" model, a critical design pattern for managing complex traffic flows between hundreds of VPCs without the operational overhead of a mesh network. We will master the Transit Gateway—the regional virtual router for your Landing Zone.

"A network is not just a collection of connections; it is the nervous system of your infrastructure. If the path is unclear, the data is lost."

Chapter 4: Inter-account Communication Theory

In Phase I, we isolated our accounts to reduce the **Blast Radius**. However, isolation without connectivity is an island, not an ecosystem. **Inter-account Communication Theory** focuses on how to allow disparate VPCs (Virtual Private Clouds) to talk to each other securely without creating a "Spaghetti Mesh" of manual connections.

The Evolution: From Peering to Hub-and-Spoke

As a Landing Zone Specialist, you must understand why we move away from traditional VPC peering for large-scale deployments:

T1. **Transit Limitations:** Standard VPC peering is non-transitive. If VPC A is peered to VPC B, and VPC B is peered to VPC C, VPC A cannot talk to VPC C through B.

T2. **The "Hub" Concept:** By creating a central "Network Account," we deploy a **Transit Gateway** that acts as a regional router. Every other account (Spoke) simply "attaches" to this hub.

T3. **Route Table Management:** Instead of managing hundreds of peering routes, we centralize routing logic in the Hub, allowing for global traffic inspection.

Pre-Class Exercises: Initializing the Network Spoke

Ex 1. **Repository Expansion:** Create /week-04-networking. Inside, create /hub-config and /spoke-attachments.

Ex 2. **CIDR Planning:** In a file titled IP_PLAN.md, list three non-overlapping CIDR blocks (e.g., 10.1.0.0/16, 10.2.0.0/16, 10.3.0.0/16) to be used for your Hub and two Spoke accounts.

Ex 3. **Network Hub Initialization:** Prepare a Terraform provider file in /hub-config that targets your dedicated "Network" or "Infrastructure" account.

Architect's Note: *Overlapping IP ranges are the death of a Landing Zone. Always use your IP_PLAN as the source of truth before running terraform apply on any VPC.*

Transit Gateway vs. VPC Peering at Scale

As a Landing Zone Specialist, you will often be asked: "Why can't we just use Peering?" The answer lies in the **complexity of n(n-1)/2**. As the number of VPCs grows, the number of peering connections required for a full mesh becomes unmanageable.

1. The Mesh Complexity (VPC Peering)

VPC Peering is a 1-to-1 relationship. If you have 10 VPCs that all need to talk to each other, you need 45 separate peering connections. Each connection requires manual acceptance and individual route table updates. Peering is excellent for high-performance, low-latency links between two specific points, but it does not scale for enterprise "East-West" traffic.

2. The Centralized Router (Transit Gateway)

Transit Gateway (TGW) acts as a regional network hub. You attach your VPCs to the TGW, and it handles the routing between them.

- **Simplified Routing:** One attachment per VPC instead of dozens of peers.
- **Transitive Routing:** VPC A can talk to VPC C through the TGW.
- **Central Inspection:** You can force all traffic through a "Security VPC" for deep packet inspection.

The Architect's Rule of Thumb:

Use **VPC Peering** for simple, low-latency connections between two accounts within the same region. Use **Transit Gateway** for any environment involving more than 3-5 VPCs or requiring cross-account centralized security.

Lab 4.1: Automated Network Hub Deployment

This lab focuses on the creation of the **Transit Gateway** and the initialization of its **Route Tables**. This hub will eventually serve as the single point of entry and exit for all inter-VPC traffic.

Task 1: Provision the Transit Gateway

Create a tgw.tf file in your /week-04-networking/hub-config directory. This code defines the router and disables "Default Route Propagation" to allow for manual, secure route control.

```
resource "aws_ec2_transit_gateway" "hub" {
  description                     = "Main Regional Network Hub"
  default_route_table_association = "disable"
  default_route_table_propagation = "disable"
  auto_accept_shared_attachments  = "enable"

  tags = {
    Name = "hex-tgw-hub-01"
  }
}
```

Task 2: Resource Access Manager (RAM) Sharing

Because the TGW lives in the Network account, we must "Share" it with the rest of the Organization so that Spoke accounts can see it and attach to it.

- [ ] Enable **Resource Access Manager** sharing for your Organization.
- [ ] Run terraform apply and capture the tgw_id output.
- [ ] Verify the TGW status is available in the Cloud Console.

> **Architectural Defense:** Why did we disable default_route_table_association? In an enterprise Landing Zone, we never want a new VPC to automatically talk to everything else. We want **Explicit Intent** for every connection.

Project 4: Building a Centralized Logging & Audit Repository

The objective of this project is to implement a **Single Source of Truth** for network forensics. You will configure VPC Flow Logs to capture traffic patterns from your Spokes and stream them into a centralized S3 bucket located in your Security or Audit account.

Project Deliverables

1. The Centralized S3 Log Bucket

Deploy a cross-account S3 bucket with a policy that allows delivery.logs.amazonaws.com to write logs from any account within your Organization. Ensure **Object Lock** or **MFA Delete** is enabled for tamper-resistance.

2. Automated VPC Flow Log Activation

Create a Terraform module that activates VPC Flow Logs for every "Spoke" VPC created in Lab 4.1. The destination must point to your centralized S3 bucket ARN.

3. The Traffic Visibility Report

In a file named LOGGING_STRATEGY.md, document how centralized logging simplifies the job of a security auditor. Explain the "Read-Only" access pattern for this account.

Submission Checklist

Cross-Account Trust	Does your S3 Bucket Policy include an Organization ID condition?
Data Immutability	Is Versioning enabled on your central log bucket?
IaC Consistency	Are your Network Hub and Logging configurations in separate Git folders?

Career Strategy: Being able to demonstrate a "Tamper-Proof Audit Trail" is the gold standard for compliance-heavy industries (FinTech, Healthcare). This project proves you can architect for compliance, not just connectivity.

WEEK 05

Shared Services, Private Link & Secure Connectivity

WEEKLY TECHNICAL SCOPE

1. **Foundation:** Service Consumption vs. Network Connectivity.
2. **Teaching Notes:** Shared VPC (GCP/AWS) vs. Private Link/Endpoint Services.
3. **Hands-on Lab:** Creating a Cross-Account Private Endpoint.
4. **Weekly Project:** Deploying a "Hidden" Internal Service Provider.

Connectivity is the first step, but security is the ultimate goal. This week, we explore how to share network resources across accounts without full peering. We will master Shared VPCs for centralized IP management and Private Link (VPC Endpoints) to allow services to talk privately, ensuring that sensitive internal data never touches the "Dirty Internet."

"The most secure connection is the one the world cannot see. True architectural mastery is making the complex feel invisible."

Chapter 5: Service Consumption Theory

In a mature Landing Zone, we distinguish between **Network Connectivity** (the ability to route packets) and **Service Consumption** (the ability to access an application). Service Consumption Theory dictates that we should minimize the number of VPCs managed by application teams to reduce "IP Sprawl" and centralize security controls.

Centralizing the Network Fabric

To achieve high-efficiency service delivery, we utilize two primary patterns:

S1. **The Shared VPC Pattern:** A central "Network Account" owns the VPC and its subnets. These subnets are "shared" with application accounts. Developers can deploy resources into these subnets but cannot change the underlying network configuration.

S2. **The Private Link Pattern:** This allows a service provider in one VPC to expose their application to a consumer in another VPC via a private endpoint. Traffic stays on the provider's backbone, bypassing the public internet entirely.

Pre-Class Exercises: Initializing Private Endpoints

Ex 1. **Repository Expansion:** Create /week-05-shared-services. Inside, create /shared-vpc-config and /endpoint-logic.

Ex 2. **Resource Tagging Policy:** Draft a TAGGING_GUIDE.md that defines how shared subnets should be tagged so that Terraform can dynamically discover them in consumer accounts.

Ex 3. **CLI Discovery:** Run a command to list all available VPC Endpoints in your current region and redirect the JSON output to available_services.json in your exercises folder.

> ***Architect's Note:*** *Shared VPCs are excellent for internal trust, but Private Link is the king of "Zero Trust" networking. If you don't need full routing access, always prefer an Endpoint over a Peer.*

Shared VPC vs. Private Link/Endpoints

Selecting the right service delivery model is critical for maintaining a clean IP space and high security. While both patterns allow cross-account communication, they solve for different organizational needs.

1. Shared VPC (Administrative Centralization)

In a Shared VPC model, a central **Host Account** owns the network infrastructure (VPC, Subnets, Route Tables). It shares specific subnets with **Service Accounts**.

- **Best For:** Organizations that want a single team to manage all IP addresses and network security groups.
- **Key Benefit:** Simplifies network topology by reducing the number of VPCs that need to be peered or attached to a Hub.

2. Private Link / Interface Endpoints (Zero-Trust)

Private Link allows you to expose a service (backed by a Network Load Balancer) to other VPCs as if it were a local IP address in the consumer's subnet.

- **Best For:** Connecting overlapping IP spaces or providing a service to accounts that should not have full network access to the provider.
- **Key Benefit:** One-way connectivity. The consumer can talk to the service, but the service cannot initiate a connection back to the consumer.

Architect's Decision Matrix:

Does the consumer need to see the entire network? ***Shared VPC****.*
Does the consumer only need to access a specific API or Database? ***Private Link****.*

Lab 5.1: Creating a Cross-Account Private Endpoint

This lab guides you through the process of "publishing" a service via an Endpoint Service and "consuming" it from a different account. This ensures that sensitive data stays within the provider's backbone.

Task 1: The Provider Configuration

In your /endpoint-logic folder, define the Endpoint Service. You must link this to a Network Load Balancer (NLB) that sits in front of your internal application.

```
resource "aws_vpc_endpoint_service" "app_provider" {
  acceptance_required        = true
  network_load_balancer_arns = [aws_lb.internal_nlb.arn]

  tags = {
    Name = "hex-private-link-service"
  }
}
```

Task 2: The Consumer Attachment

Now, switch to your "Workloads" account. You will create the Interface Endpoint that points to the Service Name generated in Task 1.

- [ ] Provision the aws_vpc_endpoint resource using the Provider's Service Name.
- [ ] Manually accept the endpoint connection request in the Provider account.

- [ ] Run a dig or nslookup on the endpoint DNS name to verify it resolves to a private IP.

> **Architectural Defense:** What is the primary security advantage of using an Interface Endpoint over a VPC Peer for an RDS Database? Consider the "Directionality" of the connection.

Project 5: Deploying a "Hidden" Internal Service Provider

The objective of this project is to build a highly secure, producer-consumer architecture. You will deploy a "Service Provider" in a restricted VPC and allow a "Consumer" in a separate account to access it via **Private Link**, ensuring the service remains invisible to the outside world.

Project Deliverables

1. The Producer Service Stack

Deploy a simple internal application (like an Nginx server or a private API) behind a Network Load Balancer (NLB) in your Infrastructure account. This application must have no Internet Gateway (IGW) attached.

2. The Private Link "Handshake" Code

Codify the Endpoint Service in Terraform. Include a whitelist of Account IDs that are allowed to discover and connect to your service. Automate the connection acceptance logic where possible.

3. The "Zero-Internet" Proof

In a file named INTERNAL_SERVICE.md, provide evidence that the consumer can reach the service, but that neither the producer nor the consumer uses a public IP address for the transaction.

Submission Checklist

Traffic Path	Is all traffic confirmed to stay on the internal provider backbone?
DNS Resolution	Does the consumer use a Private DNS name to resolve the endpoint?
IaC Modularity	Is the Endpoint Service logic separated into a reusable module?

Career Strategy: Building "Internal Marketplaces" is a key DevOps trend. Demonstrating that you can offer services across accounts securely—without the complexity of massive peering arrangements—is a high-level architectural skill.

WEEK 06

Automated Security & Detection Engineering

WEEKLY TECHNICAL SCOPE

1. **Foundation:** Continuous Monitoring & Threat Intelligence.
2. **Teaching Notes:** Delegated Administration for GuardDuty & Security Hub.
3. **Hands-on Lab:** Automating Multi-Account Security Enablement.
4. **Weekly Project:** Building an Incident Response Dashboard.

A Landing Zone is only as strong as its visibility. This week, we move from preventative guardrails to active threat detection. We will implement GuardDuty for intelligent threat monitoring and Security Hub as our "Single Pane of Glass" for compliance. By delegating administration to a dedicated Security Account, we ensure that security teams have the oversight they need without interfering with application workloads.

"Security is not a point-in-time achievement; it is a continuous state of awareness. Visibility is the first step toward resilience."

Chapter 6: Continuous Monitoring Theory

In a traditional data center, security was often a "point-in-time" exercise—an annual audit or a quarterly scan. In a Landing Zone, the speed of change makes periodic checks obsolete. **Continuous Monitoring Theory** posits that security must be an automated, always-on function that identifies threats and configuration drift the moment they occur.

The Pillars of Automated Defense

To achieve a resilient security posture, we implement two distinct but complementary layers:

M1. **Threat Intelligence (Active Defense):** Using machine learning to analyze VPC Flow Logs, CloudTrail, and DNS logs to identify malicious patterns, such as crypto-mining, credential exfiltration, or port scanning.

M2. **Compliance Monitoring (Posture Management):** Aggregating security findings against industry standards (like CIS Benchmarks or PCI-DSS) to provide a "Security Score" for the entire organization.

Pre-Class Exercises: Initializing Security Delegation

Ex 1. **Repository Expansion:** Create /week-06-security. Inside, create /detection-config and /compliance-rules.

Ex 2. **Delegated Admin Identification:** Retrieve the Account ID of your "Security" or "Audit" account. You will need this to delegate administrative control from the Management account.

Ex 3. **Baseline Audit:** Run a manual check on your "Workloads" account to see if any S3 buckets are public or if the "Root" user has an active access key. Document these as "Pre-Automation" findings.

> ***Architect's Note:*** *Centralization is the key to sanity. Never log into ten different accounts to check security status; always delegate administration to a single "Security" account to act as your Command Center.*

Delegated Administration: GuardDuty & Security Hub

Delegated Administration is the mechanism that allows a "Member Account" to act as the administrator for a specific service across the entire Organization. This follows the **Principle of Least Privilege** by ensuring the Management Account is used strictly for billing and structural changes, not daily security operations.

1. Amazon GuardDuty (Threat Detection)

When GuardDuty is delegated to the Security Account, that account gains the ability to:

- **Auto-Enable:** Automatically enable GuardDuty for any new account that joins the Organization.
- **Aggregate Findings:** View all threats (e.g., unusual API calls, known malicious IP interactions) in one central dashboard.

2. AWS Security Hub (Posture Management)

Security Hub acts as the "Aggregator of Aggregators." It ingests findings from GuardDuty, Inspector, Macie, and IAM Access Analyzer.

- **Compliance Scoring:** View a consolidated security score based on CIS Foundations or AWS Foundational Security Best Practices.
- **Central Remediation:** Trigger automated actions (like disabling an IAM key) from the delegated Security Account.

The Architect's Goal:

By the end of this week, you should never have to log into an application account to see if it is secure. If a threat exists in "Spoke-A," you should see it, analyze it, and mitigate it from "Security-Hub."

Lab 6.1: Automating Multi-Account Security

This lab demonstrates the "One-Click" enablement of security services across an entire Organization. This automation ensures that no account—old or new—ever operates without active threat detection.

Task 1: The Delegation Handshake

In your /detection-config folder, use the Management Account provider to delegate GuardDuty to your Security Account ID.

```
resource "aws_guardduty_organization_admin_account" "security_delegate" {
  admin_account_id = var.security_account_id
}

# Ensure Security Hub is also delegated
resource "aws_securityhub_organization_admin_account" "security_hub_delegate" {
  admin_account_id = var.security_account_id
}
```

Task 2: Automatic Enrollment Logic

Now, using the **Security Account** provider, configure the organization to automatically enable GuardDuty for all new members.

- [ ] Set auto_enable = true in the aws_guardduty_organization_configuration resource.
- [ ] Run terraform apply and verify that the "Workloads" account is now listed as a "Member" in the GuardDuty console of the Security Account.

- [ ] Repeat this logic for Security Hub to establish your centralized compliance dashboard.

> **Architectural Defense:** Why do we perform the delegation in the Management account but the configuration in the Security account? Think about the "Trust Relationship" between the Organization root and its members.

Project 6: Building an Incident Response Dashboard

The objective of this project is to create a **Unified Security Overview**. You will consolidate findings from GuardDuty and Security Hub into a central reporting structure, enabling rapid identification of high-severity risks across your entire Landing Zone.

Project Deliverables

1. The Centralized Findings Aggregator

Configure Security Hub to ingest findings from at least three integrated services (e.g., GuardDuty, IAM Access Analyzer, and AWS Config). Ensure cross-region aggregation is enabled if your Landing Zone spans multiple regions.

2. High-Severity Alerting Logic

Create a Terraform resource that triggers a notification (e.g., via SNS or a Lambda function) whenever a "Critical" or "High" severity finding is generated in any member account. This ensures immediate awareness of potential breaches.

3. The Security Posture Analysis

In a file named SECURITY_POSTURE.md, analyze the findings generated during the Week 6 labs. Identify the top three risks in your environment and document the automated steps taken to remediate them.

Phase II Progress Checklist

Detection Coverage	Is GuardDuty active in every account under your Organization?
Visibility Level	Can you see "Workload" findings from the "Security" account dashboard?

Alerting Accuracy	Does your notification system distinguish between "Info" and "Critical" events?

Career Strategy: Employers look for architects who can translate "Raw Data" into "Actionable Intelligence." Building this dashboard proves you can handle the operational reality of securing a multi-account enterprise.

WEEK 07

Observability, Logging & Audit Pipeline

WEEKLY TECHNICAL SCOPE

1. Foundation: Forensic Integrity & Log Aggregation Theory.

2. Teaching Notes: The Log Archive Account vs. Monitoring Accounts.

3. Hands-on Lab: Automating CloudTrail & Flow Log Centralization.

4. Weekly Project: Implementing S3 Object Lock & Tamper-Proofing.

In a multi-account environment, logs are the ultimate source of truth. This week, we architect the "Log Archive" account—a dedicated repository designed to ingest data from every corner of the Landing Zone. We focus on immutability, ensuring that even a compromised administrator cannot delete the digital footprints of their actions. This completes our Phase II networking and shared services foundation.

"Architecture is about what you build; Governance is about what you can prove. A log that cannot be altered is a truth that cannot be denied."

Chapter 7: Forensic Integrity Theory

In an enterprise Landing Zone, logs are not just for troubleshooting; they are evidence. **Forensic Integrity Theory** establishes that for logs to be valid during an incident response or a legal audit, they must be centralized, immutable, and protected by a "Chain of Custody" that prevents even the most privileged users from altering the record.

The Three Pillars of Centralized Logging

To build a "Black Box" for your cloud environment, we adhere to these core principles:

L1. **Account Isolation:** Logs must be shipped out of the account where they were generated and into a dedicated **Log Archive** account. This prevents an attacker who compromises a workload from deleting their tracks.

L2. **Write-Once-Read-Many (WORM):** Utilizing features like S3 Object Lock or immutable storage policies to ensure that once a log is written, it cannot be deleted or overwritten for a defined retention period.

L3. **Encryption with Separated Keys:** Encrypting logs using KMS keys that are managed in a separate account from the logs themselves, creating a dual-authorization requirement for decryption.

Pre-Class Exercises: Preparing the Archive

Ex 1. **Repository Expansion:** Create /week-07-logging. Inside, create /archive-setup and /trail-config.

Ex 2. **S3 Policy Drafting:** Draft a policy in bucket_policy.json that allows cloudtrail.amazonaws.com from your entire organization to put objects into a specific bucket ARN.

Ex 3. **Retention Calculator:** In RETENTION_PLAN.md, list the legal log retention requirements for your specific industry (e.g., 7 years for Finance, 1 year for General GDPR).

Architect's Note: *The Log Archive account is the only account in your Landing Zone where even the Administrator should have "Deny" on s3:DeleteObject. This is the ultimate proof of integrity.*

Log Archive vs. Monitoring Accounts

A common mistake in cloud architecture is treating logging and monitoring as the same task. While they use the same data, their **Retention**, **Access**, and **Intent** are fundamentally different.

1. The Log Archive Account (The Vault)

The Log Archive account is your **Immutable Repository**. Its primary purpose is long-term retention and audit compliance.

- **Access:** Extremely restricted. Typically, only the Audit/Compliance team has read access, and no one has delete access.
- **Retention:** Defined by legal requirements (e.g., 7 years).
- **Storage:** Optimized for cost using S3 Intelligent-Tiering or Glacier for older logs.

2. The Monitoring/Security Account (The Dashboard)

The Monitoring account (often where Security Hub lives) is your **Real-Time Analysis** center. It ingests data to trigger alerts and automated remediations.

- **Access:** Security Engineers and DevOps teams have access to view dashboards and logs for troubleshooting.
- **Retention:** Short-term (e.g., 30–90 days) for operational analysis.
- **Storage:** Higher performance (e.g., CloudWatch Logs, OpenSearch) to allow for fast querying and visualization.

Key Separation Rule:

*If you delete an alert in the Monitoring account, the underlying log in the Log Archive account must remain untouched. Monitoring is for *now*; Log Archive is for *ever*.*

Lab 7.1: Automating Log Centralization

This lab focuses on establishing a "one-way" flow of audit data. By the end of this session, your Management account will mandate logging for all child accounts, while the actual data resides in the protected **Log Archive** silo.

Task 1: Deploying the Organization Trail

Create a central_trail.tf file in your /week-07-logging/trail-config directory. This resource must be deployed using the **Management Account** provider.

```
resource "aws_cloudtrail" "org_trail" {
  name                          = "hex-org-audit-trail"
  s3_bucket_name                = var.log_archive_bucket_id
  is_organization_trail         = true
  is_multi_region_trail         = true
  include_global_service_events = true
  enable_log_file_validation    = true

  tags = {
    Environment = "Governance"
  }
}
```

Task 2: Spoke Account Flow Log Connection

Now, update your **Workloads** account networking code to stream VPC Flow Logs directly to the Log Archive S3 bucket.

[ ] Provision aws_flow_log resources for all Spoke VPCs.

- [ ] Run terraform apply and confirm that log files are appearing in the Log Archive S3 bucket under the correct account prefixes.
- [ ] Verify **Log File Validation** is enabled to ensure the digest files can detect any log tampering.

> **Architectural Defense:** Why do we use is_organization_trail = true instead of creating a trail in each account? Hint: Consider what happens if an attacker gains Administrator access in a member account and tries to disable the trail.

Project 7: Implementing S3 Object Lock & Tamper-Proofing

The objective of this project is to finalize the **Immutable Audit Vault**. You will configure your central log bucket to use "Compliance Mode" object locking, effectively placing the logs beyond the reach of any user, including the root account, for a specified duration.

Project Deliverables

1. The WORM Bucket Configuration

Update your /archive-setup Terraform code to enable object_lock_enabled = true. Define a default retention period of 365 days in **Compliance Mode**. Note: This cannot be undone once applied to an object.

2. KMS Key Separation

Create a Customer Managed Key (CMK) in the **Security** account to encrypt logs stored in the **Log Archive** account. Document the cross-account key policy that allows the Archive bucket to use this key.

3. The Compliance Validation Report

In a file named COMPLIANCE_AUDIT.md, attempt to delete a log file using the CLI and capture the "Access Denied" error. Explain why this protection is mandatory for financial or medical data residency.

Phase II Final Checklist

Networking Hub	Is the Transit Gateway routing traffic correctly between Spokes?
Security Oversight	Are GuardDuty findings visible in the Security Hub dashboard?
Forensic Integrity	Does the Log Archive account contain immutable CloudTrail logs?

Architect's Note: You have now completed the "Hardening" phase. Your Landing Zone is not just connected; it is monitored, audited, and protected against internal and external threats.

Phase II Knowledge Assessment

Assessment Overview

Questions: 18 multiple-choice questions

Time Estimate: 50-70 minutes

Passing Score: 13/18 (72%)

Coverage: Weeks 4-7 (Transit Gateway, Shared Services, Security, Logging)

1. What is the primary purpose of a Transit Gateway in a Landing Zone?

A) To provide internet access to all VPCs
B) To enable secure communication between VPCs across accounts
C) To manage DNS resolution for the organization
D) To store centralized logs from all accounts

2. In a Hub-and-Spoke network architecture, what role does the "Hub" play?

A) It contains application workloads
B) It provides centralized network services and routing
C) It stores user data and databases
D) It manages identity and access control

3. Which AWS service enables sharing of Transit Gateway across multiple accounts?

A) AWS Organizations
B) AWS Resource Access Manager (RAM)
C) AWS Systems Manager
D) AWS Config

4. What is the primary benefit of using PrivateLink for shared services?

A) It reduces network latency
B) It keeps traffic within the AWS network, avoiding internet exposure
C) It automatically scales based on demand
D) It provides built-in encryption for all traffic

5. Which service is best suited for detecting anomalous API activity across your Landing Zone?

A) AWS Config
B) AWS GuardDuty
C) AWS Inspector
D) AWS Trusted Advisor

6. What is the purpose of a "Log Archive" account in a Landing Zone?

A) To run application workloads
B) To provide centralized, immutable log storage
C) To manage user identities
D) To host shared network services

7. Which AWS service provides organization-wide visibility into security findings?

A) AWS CloudTrail
B) AWS Security Hub
C) AWS Config
D) AWS Systems Manager

8. What is CloudTrail primarily used for in a Landing Zone?

A) Cost monitoring
B) API activity auditing
C) Resource configuration management
D) Network traffic analysis

9. Which account type should contain the Security Hub master configuration?

A) Management account
B) Audit account
C) Log Archive account
D) Workload accounts

10. What is the benefit of enabling CloudTrail organization trails?

A) It reduces logging costs
B) It automatically enables logging for all current and future accounts
C) It provides real-time alerting
D) It encrypts log data automatically

11. Which service should you use to share a Transit Gateway with accounts outside your organization?

A) AWS Resource Access Manager
B) AWS Organizations
C) VPC Peering
D) Direct Connect

12. What is the primary security benefit of using GuardDuty in a multi-account setup?

A) It provides automated remediation
B) It correlates findings across accounts for broader threat detection
C) It replaces the need for Security Groups
D) It manages IAM permissions

13. Which account should have read-only access to the Log Archive S3 buckets?

A) Management account
B) Audit account
C) All workload accounts
D) No accounts should have access

14. What is the role of Transit Gateway Route Tables in network segmentation?

A) They encrypt network traffic
B) They control which VPCs can communicate with each other
C) They manage IP address allocation
D) They provide internet connectivity

15. Which service provides automated compliance monitoring and remediation in a Landing Zone?

A) AWS Config
B) AWS CloudTrail
C) AWS GuardDuty
D) AWS Security Hub

16. What is the primary purpose of VPC Flow Logs in a Network Hub?

A) To monitor application performance
B) To provide network traffic visibility for security analysis
C) To manage IP address assignments
D) To optimize routing decisions

17. Which account architecture ensures that security logs cannot be deleted by any user, including administrators?

A) Management account with SCPs
B) Separate Log Archive account with strict IAM policies
C) Workload accounts with GuardDuty
D) Audit account with Security Hub

18. What is the benefit of using organization trails over individual account trails?

A) They cost less per account
B) They provide a single source of truth for all organizational activity
C) They automatically encrypt all log data
D) They enable real-time alerting

🎯 Phase II Completion

Congratulations on completing Phase II! Your Landing Zone now has secure network connectivity and comprehensive monitoring.

Proceed to Phase III to master automation and platform governance. Your mentor will review your Phase II infrastructure before advancing.

Phase II Assessment

PHASE III

Automation & Customizations

Moving from manual configuration to programmatic scaling. This phase covers the creation of the "Account Factory"—automating the provisioning, hardening, and baseline customization of new accounts to ensure every "vended" environment is secure and compliant by default.

"True scale is achieved when the manual becomes automated, and the automated becomes invisible. We are no longer building accounts; we are building the machine that builds accounts."

WEEK 08

Automated Account Factory & Self-Service Provisioning

WEEKLY TECHNICAL SCOPE

1. Foundation: Account Vending Theory & Scalable Provisioning.

2. Teaching Notes: Terraform Organizations vs. Control Tower Account Factory.

3. Hands-on Lab: Creating an Account Vending Machine (AVM) with Terraform.

4. Weekly Project: Automating Baseline Enrollment for New Spokes.

As a Landing Zone Specialist, you must move beyond manual account creation. This week, we architect the "Account Factory"—a standardized process for "vending" new accounts through code. We will implement the logic that ensures every new environment is automatically placed in the correct Organizational Unit, tagged appropriately, and connected to our core networking and security fabrics the moment it is born.

"Scalability is the ability to repeat success without repeating effort. The Account Factory is where infrastructure becomes a product."

Chapter 8: Account Vending Theory

As an organization grows, the manual creation of accounts becomes a bottleneck and a security risk. **Account Vending Theory** treats the account as a "Product." Instead of a one-off task, we create a standardized delivery pipeline that ensures every account—whether for a Sandbox, Development, or Production environment—starts with the same baseline security, identity, and networking configurations.

The Mechanics of "Vending"

A specialist architect views account vending as a four-stage orchestration:

V1. **Request & Approval:** A developer or project lead requests an account with specific parameters (Owner, Cost Center, Environment Type).

V2. **Core Provisioning:** The infrastructure-as-code (IaC) layer triggers the creation of the account entity and places it into the correct Organizational Unit (OU).

V3. **Baseline Hardening:** Governance policies (SCPs), security services (GuardDuty), and logging (CloudTrail) are automatically enabled and locked.

V4. **Network Integration:** The account is automatically attached to the Transit Gateway hub or joined to a Shared VPC, making it "Day 1" ready for workloads.

Pre-Class Exercises: Drafting the Factory

Ex 1. **Repository Setup:** Create /week-08-account-factory. Inside, create /modules/account-vending-machine and /accounts-inventory.

Ex 2. **Variable Definition:** Create a variables.tf that defines the required metadata for any new account (e.g., email, account_name, environment_tag).

Ex 3. **Governance Check:** Identify which Service Control Policies (SCPs) from Week 3 must be automatically attached to any new account vended by the factory.

> ***Architect's Note:*** *The goal is not just "Speed," but "Consistency." A factory-vended account is safe because it is impossible for a human to forget a security step in a coded pipeline.*

Terraform Organizations vs. Control Tower

Deciding between a custom-coded factory and a managed service is a pivotal moment for any architect. Both paths achieve automation, but they differ significantly in their long-term maintenance and customizability.

1. AWS Control Tower (Managed Vending)

Control Tower provides an out-of-the-box Account Factory. It is ideal for organizations that want the cloud provider to manage the underlying orchestration of OUs, accounts, and mandatory guardrails.

- **Pros:** Simplified setup; managed guardrails; integrated dashboard for compliance status.
- **Cons:** Limited customization; can be difficult to "undo" certain managed configurations; potentially higher complexity when integrating with 3rd party tools.

2. Terraform-Driven AVM (Custom Vending)

A custom Account Vending Machine (AVM) built in Terraform gives you complete control over the provisioning logic.

- **Pros:** Complete flexibility; multi-cloud compatibility (conceptually); fits perfectly into existing CI/CD pipelines.
- **Cons:** High maintenance; your team is responsible for the integrity of the provisioning code and guardrail enforcement.

The Decision Metric:

If your client requires **Standard Compliance** with minimal overhead, choose **Control Tower**. If your client requires **Deep Customization** and uses complex Infrastructure-as-Code patterns, build a **Terraform AVM**.

Lab 8.1: Creating a Custom Terraform AVM

This lab guides you through building a reusable Terraform module that handles the lifecycle of a cloud account. This module is the cornerstone for Hexadigitall's automation strategy, ensuring every environment is a "Clone of the Best."

Task 1: The Account Provisioning Module

Navigate to /week-08-account-factory/modules/account-vending-machine and create a main.tf. This code creates the account and places it in the designated OU.

```
resource "aws_organizations_account" "vended_account" {
  name      = var.account_name
  email     = var.account_email
  parent_id = var.target_ou_id

  # This allows the Management account to manage the new account
  role_name = "OrganizationAccountAccessRole"

  tags = {
    ManagedBy   = "Hexadigitall-AVM"
    Environment = var.env_tag
  }
}
```

Task 2: Testing the Vending Logic

In your /accounts-inventory folder, call the module to create a new "Sandbox" account. Use a unique email address (many providers allow email+suffix@domain.com).

- [ ] Run terraform plan and confirm that one account resource is being created.
- [ ] Run terraform apply and monitor the Organizations console.
- [ ] Once the account is active, log in via SSO to verify the OrganizationAccountAccessRole is present.

> **Architectural Defense:** What happens if you run terraform destroy on an aws_organizations_account? *Crucial Note:* In AWS, deleting the resource from Terraform only removes it from the state/organization; it does not "delete" the account itself. This requires manual closure or specific API calls.

Project 8: Automating Baseline Enrollment

The objective of this project is to create an **Autonomic Landing Zone**. You will extend your AVM module to perform "Post-Provisioning logic." This means the code that creates the account will now also reach into that account to set up its initial defenses.

Project Deliverables

1. The "Post-Provision" Handshake

Update the AVM module to include a second Terraform provider that assumes the OrganizationAccountAccessRole in the newly created account. Use this provider to enable a specific "Baseline" resource (e.g., an S3 Public Access Block at the account level).

2. Governance Alignment

Codify the attachment of the **Deny-Leaving-Organization** SCP to the vended account. Ensure the AVM module requires a cost_center tag before it will execute, enforcing financial accountability from Day 1.

3. The Provisioning Log

Create a VENDING_REPORT.md that tracks the execution time of your factory. Document the sequence of events from "Resource Created" to "Security Baseline Applied."

Success Criteria

Zero-Touch Config	Can you create a fully hardened account with a single 'terraform apply'?
Tagging Compliance	Does the new account automatically inherit mandatory tags?

State Integrity	Is the vended account's metadata correctly stored in the Management state?

Architect's Insight: In Hexadigitall's world, an account that isn't baselined isn't "live." By forcing baseline enrollment into the AVM, you eliminate the "Security Gap" that occurs between account creation and manual hardening.

WEEK 09

Account Customizations, Post-Deployment Controls & Drift Guardrails

WEEKLY TECHNICAL SCOPE

1. **Foundation:** Idempotent Customization & Lifecycle Hooks.
2. **Teaching Notes:** Cloud-Init vs. Terraform Provider Orchestration.
3. **Hands-on Lab:** Deploying Regional Baselines & IAM Custom Roles.
4. **Weekly Project:** The "Environment-Aware" Customizer.

Creation is only half the battle. A production account requires different "DNA" than a sandbox environment. This week, we master the art of Post-Deployment Logic—using Terraform hooks and provider aliasing to inject environment-specific configurations into vended accounts. We move beyond universal baselines to create tailored infrastructure that respects the unique requirements of the workloads it will host.

"Standardization provides the floor, but customization provides the ceiling. Architectural excellence is found in the balance between the two."

Chapter 9: Idempotent Customization Theory

In a mature Landing Zone, customization is not a one-time script; it is a continuous state. **Idempotent Customization Theory** dictates that our post-deployment logic must be able to run a thousand times and result in the same configuration every time, without side effects or errors.

The Customization Lifecycle

To scale **Hexadigitall Tech** deployments, we categorize customizations into three distinct tiers:

T1. **Global Baselines:** Mandatory settings for every account (e.g., IAM password policies, disabling unused regions).

T2. **OU-Specific Logic:** Configurations unique to a business unit (e.g., specialized VPC endpoints for the Data Science team).

T3. **Environment Overrides:** Tuning resources based on risk (e.g., multi-region failover for 'Prod', single-zone for 'Dev').

Pre-Class Exercises: Mapping the Logic

Ex 1. **Repository Extension:** Create /week-09-customizations. Inside, create /modules/baseline-roles and /logic-gateways.

Ex 2. **Drafting the "DNA":** In DNA_MATRIX.md, list three resources that should exist in a "Production" account but should be absent in a "Sandbox" account to save costs.

Ex 3. **Alias Practice:** Research the Terraform alias keyword. Write a snippet that defines two different providers for the same AWS account using different IAM roles.

> ***Architect's Note:*** *Never use a script to "push" configuration. Use Terraform to "declare" the state. If a human manually changes a setting, the next Terraform run should quietly and firmly change it back.*

Cloud-Init vs. Terraform Orchestration

To build a cohesive Landing Zone, we must distinguish between **Infrastructure Bootstrapping** (the account) and **OS Bootstrapping** (the instance).

1. Cloud-Init (The "Inside-Out" Approach)

Cloud-init is a multi-distribution package that handles early initialization of a cloud instance.

- **Scope:** Local to the OS (users, packages, files).
- **Use Case:** Joining a node to a Kubernetes cluster or installing Nginx on first boot.

2. Terraform Orchestration (The "Outside-In" Approach)

This involves using Terraform providers to reach into a newly created account and configure the API-level resources.

- **Scope:** Account-wide (IAM Roles, S3 buckets, VPCs).
- **Use Case:** Deploying the **Hexadigitall Tech** security baseline or cross-account peering.

> **The Golden Rule of Customization:**
> *Use Terraform to build the **Container** (the account and network). Use Cloud-Init (or Ansible/Puppet) to configure the **Contents** (the server*

> *software).*

Lab 9.1: Regional Baselines & Custom IAM

A Landing Zone architect must ensure that no region is left unmonitored. This lab utilizes Terraform for_each loops to deploy resources across multiple regions simultaneously and establishes a cross-account trust for auditing.

Task 1: The Multi-Region EBS Baseline

Security best practice requires that all EBS volumes be encrypted at rest. We will enforce this account-wide across all active regions.

```
# Example of enforcing EBS encryption by default
resource "aws_ebs_encryption_by_default" "enforced" {
  enabled = true
}
```

Task 2: Deploying the 'HexaAuditor' Role

This task establishes a role that allows the Security account to assume a read-only position in the new Spoke account for compliance checks.

- [ ] Define a **Trust Policy** that allows arn:aws:iam::[Security_Account_ID]:root to assume the role.
- [ ] Attach the ReadOnlyAccess AWS managed policy to the new HexaAuditor role.
- [ ] Use a Terraform provider with an alias to deploy this resource specifically into your "Sandbox" account.

> **Architectural Defense:** Why do we prefer creating a custom HexaAuditor role instead of just using the default OrganizationAccountAccessRole for security audits? (Hint: Think about the *Scope of Permissions*).

Project 9: The "Environment-Aware" Customizer

The objective of this project is to implement **Conditional Logic** within your Terraform AVM. You will build a system that detects the account type (e.g., 'Sandbox' vs. 'Production') and applies a customized set of resources and restrictions based on that context.

Project Deliverables

1. Conditional Baseline Logic

Update your AVM module to use Terraform count or for_each with conditional logic. For example: Enable **S3 Versioning** and **MFA Delete** in 'Production' accounts, but keep them optional for 'Dev' to reduce complexity.

2. Regional Lockdown Customization

Implement a "Region Whitelist" that varies by environment. 'Sandbox' accounts should be restricted to a single region (to prevent unexpected costs), while 'Production' should have a multi-region baseline enabled by default.

3. The Customization Matrix

Create a CUSTOMIZATION_LOG.json file that maps account IDs to their applied "DNA" profiles. This file serves as your proof that the AVM successfully differentiated between account types during deployment.

Success Criteria

Profile Differentiation	Do 'Dev' and 'Prod' accounts show distinct configurations after a single run?
Resource Idempotency	Does rerunning the plan result in "No changes" for both account types?

Logic Scalability	Can you add a third account type (e.g., 'Staging') by changing just one variable?

Hexadigitall Strategy: In professional consulting, we don't build one-off environments. We build **templates of intent**. By making your AVM "environment-aware," you ensure the business logic is baked into the code, not the manual checklist.

WEEK 10

Platform Delivery with AFT & Terraform Automation

WEEKLY TECHNICAL SCOPE

1. **Foundation:** GitOps for Account Provisioning.
2. **Teaching Notes:** AFT Architecture & Pipeline Orchestration.
3. **Hands-on Lab:** Configuring the AFT Request Pipeline.
4. **Weekly Project:** Deploying Global Customizations via AFT.

Enterprise Extension: Change Governance & Approval Workflows
Design change control procedures for account factory updates (separate from deployment mechanics). Document approval decision trees, escalation paths, and rollback authorization criteria. Create a change log template and playbook for communicating infrastructure changes to stakeholders.

Manual Terraform runs and custom AVMs take you far, but for high-velocity enterprises, we need **Account Factory for Terraform (AFT)**. This week, we dismantle the "black box" of AFT to understand how it uses GitOps principles to trigger account creation. We will explore how a single git commit can trigger a series of Step Functions and Lambda workers to provision, harden, and customize an account in a fully decoupled, resilient pipeline.

"Complexity is the enemy of scale. AFT turns the complex orchestration of account management into a simple, predictable stream of code."

Chapter 10: GitOps for Account Provisioning

As **Hexadigitall Technologies** moves toward enterprise-grade scaling, we must eliminate the "human-in-the-loop" for infrastructure changes. **GitOps for Account Provisioning** is a paradigm where the intended state of the entire multi-account environment is defined in Git. When a change is merged, the **Account Factory for Terraform (AFT)** automatically reconciles the physical cloud state with the code.

The AFT Operational Model

AFT introduces a decoupled architecture that separates the **request** from the **execution**:

G1. **The Request Repository:** A simple list of accounts (JSON/HCL). A pull request to this repo is the only way to "vending" a new account.

G2. **The Global Customization Pipeline:** Automated logic that applies the "Hexa-Baseline" (Security Hub, GuardDuty, IAM Roles) to every account AFT manages.

G3. **The Account Customization Pipeline:** Environment-specific logic (e.g., adding a specific VPC for 'Payment-Processing' accounts) triggered automatically after the baseline is set.

Pre-Class Exercises: Architecting the Flow

Ex 1. **Repository Planning:** Create /week-10-aft. Inside, create a folder structure mimicking the four AFT repositories: aft-management, aft-account-requests, aft-global-customizations, and aft-account-customizations.

Ex 2. **Service Limit Audit:** Use the CLI to check the current account limit for your Organization. AFT will fail if the Organization hits its account quota during vending.

Ex 3. **Webhook Strategy:** Draft a WEBHOOK_LOGIC.md explaining how a merge to the 'main' branch in GitHub triggers a CodePipeline in the AFT Management account.

> ***Architect's Note:*** *In an AFT-managed environment, your "Terraform Plan" happens in a Pull Request, and your "Apply" happens in a Merge. This is the highest form of auditability for a Landing Zone Specialist.*

AFT Architecture & Orchestration

AFT is not a single tool; it is a **Framework of Frameworks**. It manages the lifecycle of accounts by coordinating between the Management account (where Organizations lives) and a dedicated AFT Management account (where the automation runs).

1. The Four-Repo Strategy

To maintain separation of concerns, AFT requires four distinct Git repositories:

- **Account Requests:** The triggers for new account creation.
- **Global Customizations:** Resources every account must have.
- **Account Customizations:** Code applied only to specific accounts or OUs.
- **AFT Management:** The code that deploys the AFT engine itself.

2. The Event-Driven Heartbeat

When a request is merged, AFT doesn't just run terraform apply. It triggers a **Step Function** that:

- Verifies the account doesn't already exist.
- Calls Control Tower to vend the account.
- Waits for the account to be "Ready" before triggering the customization pipelines.

Architectural Insight:

AFT handles the "Waiting Game." Because account creation is an asynchronous process that can take 30+ minutes, AFT's state-machine architecture ensures that the Terraform customization phase doesn't start until the account API is fully active.

Lab 10.1: The GitOps Request Flow

In an AFT environment, we do not call aws organizations create-account. Instead, we commit a declaration to the **Account Requests** repository. AFT watches this repo and triggers a Step Function to fulfill the request.

Task: Defining the HCL Request

In your /aft-account-requests directory, create sandbox-account.tf. This file defines the metadata AFT needs to orchestrate the account.

```
module "sandbox_account_01" {
  source = "./modules/aft-account-request"

  control_tower_parameters = {
    account_email              = "devops+aft01@hexadigitall.com"
    account_name               = "hex-aft-sandbox-01"
    managed_organizational_unit = "Sandbox"
    sso_user_email             = "admin@hexadigitall.com"
    sso_user_first_name        = "Tech"
    sso_user_last_name         = "Poet"
  }

  account_tags = {
    "Project"     = "Landed-App"
    "CostCenter"  = "1002"
    "Provisioner" = "AFT"
  }
}
```

The Trigger Logic

[ ] Validate the HCL syntax using terraform validate.

- [ ] Simulate a **Git Push**. In a real AFT setup, this merge to main would trigger the CodePipeline in the AFT Management account.

> **Architect's Challenge:** Why does AFT require the SSO User's First and Last name? Hint: Think about how Control Tower integrates with AWS IAM Identity Center during the account factory process.

Lab 10.2: Global Baseline Injection

The **aft-global-customizations** repository is the engine of standardization. Every resource defined here will be applied to every AFT-vended account. This ensures our **Hexadigitall** security baseline is never optional.

Task: Defining the Global Baseline

In /aft-global-customizations/terraform/main.tf, define an account-level baseline that locks down sensitive configurations.

```
# Enforce S3 Public Access Block at the account level
resource "aws_s3_account_public_access_block" "global_block" {
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

# Create a local 'HexaAdmin' role for break-glass scenarios
resource "aws_iam_role" "hexa_breakglass" {
  name = "HexaBreakGlassAdmin"
  assume_role_policy = data.aws_iam_policy_document.trust_mgmt.json
}
```

The Orchestration Verification

- [ ] Draft the api_helpers.py (optional) if you need custom Python logic to run alongside Terraform.
- [ ] Map out the **Execution Order**: Account Vending → Global Customizations → Account Specific Customizations.

> **Architect's Note:** AFT Global Customizations are applied using a **Terraform backend** that lives in the AFT Management account. This means you don't need to manage state files for individual vended accounts; AFT handles the mapping for you.

Project 10: Deploying Global Customizations via AFT

The objective of this project is to implement a **Multi-Account Security Baseline** that is automatically applied to every account vended through AFT. You will transition from "One-by-One" account setup to "Fleet-Wide" governance.

Project Deliverables

1. The Global Security DNA

Update the aft-global-customizations repository to include a mandatory **AWS Config** recorder and an **IAM Access Analyzer** instance in every region. These resources must be deployed automatically upon account provisioning.

2. The Account-Specific Override

Configure the aft-account-customizations repo to detect an account tagged with Workload: Payment and automatically provision a dedicated **CloudHSM** cluster or a KMS Key with specific regulatory policies.

3. The AFT Health Dashboard

Create a PROVISIONING_FLOW.md that documents the status of the AFT Step Functions. Capture a successful execution trace that shows the transition from "Account Created" to "Global Customization Complete."

Success Criteria

Standardization	Are the security resources present in a newly vended account without manual intervention?
Pipeline Integrity	Does a failure in customization trigger an AFT notification/alert?
GitOps Traceability	Can you link every resource in the Spoke account back to a specific commit in Git?

> **Architect's Final Thought:** Automation is not about doing things faster; it is about doing things correctly every single time. With AFT, you have moved **Hexadigitall** from a craft-shop to a factory.

WEEK 11

Standardization, Blueprints & Service Catalog Onboarding

WEEKLY TECHNICAL SCOPE

1. **Foundation:** Policy-Driven Governance in Multi-Cloud.
2. **Teaching Notes:** Azure Blueprints vs. AWS Service Catalog.
3. **Hands-on Lab:** Architecting an Azure Landing Zone Template.
4. **Weekly Project:** The "Service-in-a-Box" Catalog Deployment.

Enterprise Extension: Workload Onboarding
Build a workload onboarding playbook that guides teams from blueprint selection through deployment verification. Document governance hand-off, compliance validation checklists, and portfolio artifact requirements for each catalog service.

As **Hexadigitall** scales, clients will demand cloud-agnostic excellence. This week, we pivot to the Microsoft ecosystem to master **Azure Blueprints**. We will explore how to bundle resource groups, policies, role assignments, and ARM templates into a single, repeatable "package." We also dive into the **Service Catalog**—the portal that allows developers to self-provision pre-approved, compliant infrastructure without needing administrative privileges.

"Consistency is the companion of authority. When infrastructure is standardized, security becomes a silent partner rather than a noisy

obstacle."

Chapter 11: Policy-Driven Governance

In a multi-cloud enterprise like **Hexadigitall Technologies**, the definition of "compliance" must remain constant, even as the underlying API changes. **Policy-Driven Governance** is the practice of codifying organizational rules (e.g., "all data must be encrypted," "no public buckets") into machine-readable policy engines that prevent non-compliant resources from ever being created.

The Hierarchy of Standardized Environments

To achieve "Standardization-as-Code," we leverage the Azure Management Hierarchy:

P1. **Management Groups:** The equivalent of AWS OUs. Policies assigned here are inherited by all child subscriptions, ensuring global guardrails.

P2. **Azure Policy (Guardrails):** Real-time enforcement. Unlike SCPs which only "Deny," Azure Policy can "Audit," "Deny," or even "DeployIfNotExist" to automatically remediate drift.

P3. **Blueprints (The Template):** A container for resource groups, role assignments, and policy definitions. It ensures that a new environment isn't just "governed," but fully "composed" of the right parts.

Pre-Class Exercises: Mapping Azure to AWS

Ex 1. **Comparison Table:** Create /week-11-standardization/CLOUD_MAPPING.md. List AWS services (Control Tower, SCP, AVM) and their Azure equivalents.

Ex 2. **Policy Drafting:** Write a pseudo-code policy that denies the creation of any storage account if supportsHttpsTrafficOnly is set to false.

Ex 3. **Hierarchy Design:** Sketch a Management Group structure for a company with "Finance" and "Engineering" departments, each having "Dev" and "Prod" subscriptions.

> ***Architect's Note:*** *In Azure, governance is a "First-Class Citizen." While AWS layers governance on top of accounts, Azure builds it into the fabric of the Management Group hierarchy from the ground up.*

Azure Blueprints vs. AWS Service Catalog

Standardization is about control without the bottleneck. We achieve this by moving from "Tickets" to "Self-Service," using templates that have security and compliance baked in from the start.

1. Azure Blueprints (The Orchestrator)

Azure Blueprints allows you to define a repeatable set of Azure resources that implements and adheres to an organization's standards.

- **Philosophy:** Environment-focused. It creates the "Subscription Baseline."
- **Locking:** Features "Resource Locking" to prevent even administrators from deleting critical governance infrastructure.

2. AWS Service Catalog (The Vending Machine)

Service Catalog allows organizations to create and manage catalogs of IT services that are approved for use on AWS.

- **Philosophy:** Component-focused. It creates the "Application Lego."
- **Permissions:** Uses "Launch Constraints" to allow users to deploy resources they wouldn't normally have IAM permissions to create.

The Architect's Synthesis:

For **Hexadigitall**, we use **Blueprints** to set up the foundation of an Azure Subscription (Networking, RBAC, Policies). We use **Service Catalog** in AWS to give developers a "Menu" of pre-hardened databases and compute clusters that comply with our Phase II security rules.

Lab 11.1: The Azure Hierarchy

Azure governance starts at the **Management Group** level. We will create a hierarchy that separates "Production" from "Sandbox" and assigns a policy to ensure all data is kept within a specific geographic region.

Task: Defining the Bicep Blueprint

In your /week-11-standardization/azure-baseline directory, create hierarchy.bicep. This script builds the organizational containers.

```
targetScope = 'tenant'

resource hexadigitalIMG 'Microsoft.Management/managementGroups@2021-04-01'
= {
  name: 'hex-org'
  properties: {
    displayName: 'Hexadigital Technologies'
  }
}

resource productionMG 'Microsoft.Management/managementGroups@2021-04-01'
= {
  name: 'hex-prod'
  parent: hexadigitalIMG
  properties: {
    displayName: 'Hexa-Production'
  }
}
```

The Deployment Command

[ ] Login via az login and ensure you have 'Owner' rights at the Tenant Root.

- [ ] Run az deployment tenant create --location EastUS --template-file hierarchy.bicep.
- [ ] Verify in the Azure Portal that the hierarchy is reflected under 'Management Groups'.

> **Architect's Challenge:** Why did we set the targetScope to tenant instead of subscription? Hint: Think about where Management Groups "live" in the Azure resource model.

Lab 11.2: The Standardized Spoke VNET

Connectivity must be consistent. This lab creates a "Network Blueprint" that ensures every new Azure Spoke follows the **Hexadigitall** security posture for traffic isolation.

Task: Implementing the Networking Logic

```
resource vnet 'Microsoft.Network/virtualNetworks@2021-05-01' = {
  name: 'hex-spoke-vnet'
  location: resourceGroup().location
  properties: {
    addressSpace: {
      addressPrefixes: [ '10.1.0.0/16' ]
    }
    subnets: [
      {
        name: 'WebSubnet'
        properties: { addressPrefix: '10.1.1.0/24' }
      }
      {
        name: 'AppSubnet'
        properties: { addressPrefix: '10.1.2.0/24' }
      }
    ]
  }
}
```

Verification Steps

[ ] Deploy the Bicep template into a dedicated Resource Group in the 'Sandbox' subscription.

- [ ] Check the 'Effective Security Rules' on the subnets to ensure the default "Deny-All" inbound from the Internet is active.

> **Architect's Note:** By standardizing the VNET structure, you simplify the "Phase IV" operations where you will need to monitor traffic across hundreds of spokes. If the subnets are named the same everywhere, the scripts are easier to write.

Project 11: The "Service-in-a-Box" Catalog

The objective of this project is to implement a **Standardized Product Portfolio**. You will package a complex architecture—such as a three-tier web app with a hardened database—into a single, clickable product in the AWS Service Catalog or Azure Managed Applications portal.

Project Deliverables

1. The Hardened Product Template

Create a CloudFormation or ARM template for a "Hexa-Standard Database." This product must automatically enable encryption, multi-AZ failover, and restricted security group ingress from the 'AppSubnet' only.

2. Launch Constraint Orchestration

Configure a **Launch Role** that allows the Service Catalog to provision resources on behalf of a developer who lacks direct 'CreateResource' permissions. This is the cornerstone of "least-privilege" self-service.

3. The Self-Service Portal Guide

Draft a CATALOG_INVENTORY.md that lists the available "Standard Products." For each, include the cost-per-hour estimate and the specific Phase II security rules it satisfies.

Success Criteria

Zero-Privilege Deployment	Can a 'Read-Only' user successfully deploy the product via the Catalog?
Compliance Verification	Do the provisioned resources trigger any GuardDuty or Azure Policy alerts?
Cost Accountability	Are all catalog-provisioned resources tagged with the correct user metadata?

> **Architect's Insight:** You have now completed **Phase III**. You've built the factory, the blueprints, and finally, the shop front. Your Landing Zone is no longer a static project; it is a living service.

Phase III Knowledge Assessment

Assessment Overview

Questions: 18 multiple-choice questions

Time Estimate: 50-70 minutes

Passing Score: 13/18 (72%)

Coverage: Weeks 8-11 (AFT, Terraform, Service Catalog, Drift Management)

1. What is the primary purpose of AWS Control Tower Account Factory for Terraform (AFT)?

A) To manage IAM users across accounts
B) To provide automated account provisioning and customization
C) To monitor resource usage and costs
D) To create network connectivity between accounts

2. Which AFT repository is responsible for applying account-specific customizations?

A) aft-global-customizations
B) aft-account-customizations
C) aft-account-requests
D) aft-management

3. What is "Infrastructure Drift" in the context of Terraform?

A) Network latency between regions
B) Manual changes to resources not managed by Terraform
C) Automatic scaling of resources
D) Migration of resources between accounts

4. Which AWS service allows users to deploy pre-approved infrastructure without IAM permissions?

A) AWS Service Catalog
B) AWS Config
C) AWS Systems Manager
D) AWS Organizations

5. What is the role of "Launch Constraints" in AWS Service Catalog?

A) To limit the number of products that can be launched
B) To specify which IAM role performs the actual resource creation
C) To define cost limits for product deployments
D) To restrict product launches to specific regions

6. Which Terraform feature helps prevent multiple engineers from applying changes simultaneously?

A) Terraform workspaces
B) State locking with DynamoDB
C) Remote state
D) Terraform Cloud

7. What is the primary benefit of using AWS Config Rules for drift detection?

A) They automatically remediate non-compliant resources
B) They provide continuous monitoring of resource configurations
C) They manage IAM permissions
D) They create backups of resources

8. Which AFT component manages the account provisioning pipeline?

A) AWS CodePipeline
B) AWS Step Functions
C) AWS Lambda
D) AWS EventBridge

9. What is a "Product" in AWS Service Catalog terminology?

A) A pricing tier
B) A CloudFormation template or Terraform configuration
C) A user permission level
D) A compliance requirement

10. Which repository triggers the creation of new accounts in AFT?

A) aft-global-customizations
B) aft-account-requests
C) aft-account-provisioning-customizations
D) aft-management

11. What is the purpose of "Portfolio" in AWS Service Catalog?

A) To track costs
B) To group and share related products with specific users
C) To define network architectures
D) To manage compliance rules

12. Which AWS service can automatically remediate non-compliant resources?

A) AWS Config
B) AWS Systems Manager
C) AWS Lambda
D) AWS Step Functions

13. What is "GitOps" in the context of Landing Zone management?

A) Using Git for version control only
B) Treating Git as the source of truth for infrastructure state
C) Managing user permissions through Git
D) Storing logs in Git repositories

14. Which AFT repository contains customizations applied to all accounts?

A) aft-account-requests
B) aft-global-customizations
C) aft-account-customizations
D) aft-management

15. What is the benefit of using Terraform modules in a Landing Zone?

A) They reduce code duplication
B) They automatically scale resources
C) They manage user permissions
D) They create backups

16. Which service provides "Continuous Compliance" monitoring in a Landing Zone?

A) AWS Trusted Advisor
B) AWS Config
C) AWS Cost Explorer
D) AWS Personal Health Dashboard

17. What is the role of "State Management" in Terraform?

A) Managing user sessions
B) Tracking the current state of infrastructure
C) Controlling access to resources
D) Monitoring performance metrics

18. Which AWS service enables self-service infrastructure deployment with governance?

A) AWS Control Tower
B) AWS Service Catalog
C) AWS Organizations
D) AWS Resource Groups

🎯 **Phase III Completion**

Congratulations on completing Phase III! Your Landing Zone now has automated delivery and governance.

Proceed to Phase IV for lifecycle management and capstone delivery. Your mentor will review your automation pipelines before advancing.

Phase III Assessment

PHASE IV

Lifecycle & Capstone

The final frontier of the Landing Zone Specialist. In this phase, we master the art of maintaining a "Day 2" environment—detecting configuration drift, managing the lifecycle of retired accounts, and ensuring the platform evolves without breaking. Everything leads to the Capstone: The complete Hexadigitall Reference Architecture.

"Architecture is not a destination; it is a persistent state of alignment. The best Landing Zones are not those that never change, but those that change with intention."

WEEK 12

Compliance Validation, Audit Evidence & Lifecycle Guardrails

WEEKLY TECHNICAL SCOPE

1. **Foundation:** Detecting Configuration Drift at Scale.
2. **Teaching Notes:** Account Decommissioning & Data Retention.
3. **Hands-on Lab:** Automating Drift Remediation with AWS Config.
4. **Weekly Project:** The "Landing Zone Health" Dashboard.

Enterprise Extension: Compliance Validation & Audit Evidence
Build control evidence packages that demonstrate organizational policy adherence. Create audit readiness checklists (SOC 2, ISO 27001, PCI-DSS) and document how your landing zone architecture meets regulatory requirements.

The masterpiece is finished, but now it must live. This week, we address the reality of "Day 2" operations. We will build mechanisms to detect when an administrator bypasses our automation to make manual changes—known as **Drift**. We also establish the protocols for the end of the line: how to safely decommission accounts, scrub data, and release resources back to the organization without leaving security holes behind.

"A building that cannot be maintained is a ruin in waiting. A Landing Zone that cannot handle drift is a security incident in progress."

Chapter 12: Detecting Configuration Drift

In a perfect world, all changes to a Landing Zone go through a Pull Request. In the real world, emergency hotfixes, manual console overrides, and "shadow IT" create **Configuration Drift**. Left unmanaged, drift weakens your security posture and breaks your automation pipelines.

The Mechanics of Drift Awareness

At **Hexadigitall**, we view drift detection as a continuous audit loop comprising three distinct layers:

D1. **Terraform State Reconciliation:** Using CI/CD schedules to run terraform plan periodically. If the plan isn't empty, drift exists.

D2. **Resource-Level Compliance:** Leveraging **AWS Config** or **Azure Policy** to monitor individual resource properties (e.g., an S3 bucket becoming public) in real-time.

D3. **Management-Level Drift:** Detecting when a member account is moved to a different OU or when a Service Control Policy (SCP) is detached by a compromised Management-account user.

Pre-Class Exercises: Identifying the Gap

Ex 1. **Manual Override Test:** Manually change a tag on a resource in your Sandbox account via the Console. Run terraform

plan and capture how Terraform identifies this as drift.

Ex 2. **Config Rule Research:** Identify three "AWS Config Managed Rules" that directly correspond to the security baselines we built in Phase II.

Ex 3. **Drift Severity Matrix:** Create DRIFT_SEVERITY.md. Categorize drift (e.g., "Missing Tag" = Low, "Public S3" = Critical) and define response times for each.

Architect's Note: *Drift is not a failure of the engineer; it is a feature of the cloud. The goal isn't to prevent 100% of drift—it's to ensure that 100% of drift is visible and manageable.*

Decommissioning & Data Retention

Closing an account is more than clicking a button. At the enterprise scale of **Hexadigitall**, it involves a multi-step orchestration to protect intellectual property and satisfy compliance audits.

1. The Offboarding Protocol

Before an account is formally closed in AWS Organizations, the following "Deep Clean" must occur:

- **Resource Purge:** Deleting all non-storage resources (EC2, RDS, Lambda) to stop billing immediately.
- **Final Snapshot:** Moving critical database snapshots and S3 data to the **Log Archive** or a dedicated "Cold Storage" account.
- **Network Detachment:** Removing Transit Gateway attachments and Peering connections to prevent traffic from hitting a void.

2. The Retention "Black Hole"

Legal requirements often mandate that even if an account is closed, its logs and primary data must exist for years.

- **S3 Glacier Deep Archive:** The destination for retired account data. Cost-effective for 7-10 year retention.
- **Immutable Backups:** Ensuring the final state of the account is captured in an AWS Backup vault with "Vault Lock" enabled.

Operational Risk Warning:

Never close a Management Account or an account that owns a **Service Quota** shared with the org. Always identify cross-account dependencies (KMS keys, Shared Subnets) before pulling the plug.

Lab 12.1: Defining the Detective Rule

We will use Terraform to deploy an **AWS Config Managed Rule**. This rule will act as our "eyes" in the Spoke account, flagging any S3 bucket that does not have server-side encryption enabled.

Task: Deploying the Config Rule

```
resource "aws_config_config_rule" "s3_encryption_check" {
  name = "s3-bucket-server-side-encryption-enabled"

  source {
    owner            = "AWS"
    source_identifier = "S3_BUCKET_SERVER_SIDE_ENCRYPTION_ENABLED"
  }

  depends_on = [aws_config_configuration_recorder.main]
}
```

Verification Steps

- [ ] Create an unencrypted S3 bucket in the Console named hex-drift-test-bucket.
- [ ] Navigate to the AWS Config dashboard and wait for the **Non-compliant** status to appear.

Lab 12.2: Closing the Remediation Loop

We will now configure **Active Remediation**. This ensures that even if an administrator manually disables encryption, the Landing Zone will "self-heal" within minutes.

Task: Configuring the Remediation Action

```
resource "aws_config_remediation_configuration" "s3_auto_fix" {
  config_rule_name = aws_config_config_rule.s3_encryption_check.name
  resource_type    = "AWS::S3::Bucket"
  target_type      = "SSM_DOCUMENT"
  target_id        = "AWS-ConfigureS3BucketEncryption"

  parameter {
    name         = "BucketName"
    resource_value = "RESOURCE_ID"
  }

  parameter {
    name         = "SSEAlgorithm"
    static_value = "AES256"
  }
}
```

The Final "Drift" Test

- [ ] Attempt to disable encryption on hex-drift-test-bucket.
- [ ] Monitor the **SSM Automation Execution** console.

- [ ] Verify the bucket encryption is re-enabled automatically without manual intervention.

> **Architect's Note:** While auto-remediation is powerful, use it carefully. Automatically deleting a non-compliant resource (like an RDS instance) could cause data loss. Always prefer **Remediation by Configuration** (fixing the setting) over **Remediation by Destruction**.

Project 12: The "Landing Zone Health" Dashboard

The objective of this project is to aggregate disparate security and operational data into a **Centralized Health Monitor**. You will use Amazon CloudWatch and AWS Config Aggregators to provide a real-time view of your environment's posture.

Project Deliverables

1. The Config Aggregator

Deploy an **AWS Config Multi-Account Aggregator** in your Audit account. It must pull compliance data from all Sandbox and Production accounts to show a single "Compliance Score."

2. The Drift Alarm System

Configure a CloudWatch Metric Filter that triggers an SNS notification to the devops@hexadigitall.com alias whenever an SSM Auto-Remediation task is executed.

3. The Decommissioning Script

Draft a PURGE_SPOKE.py script (or Boto3 equivalent) that lists all "vulnerable" resources in an account before it is closed, serving as a pre-flight checklist for the architect.

Success Criteria

Centralized View	Can you see the compliance status of 'Account B' while logged into 'Audit Account'?
Drift Alerting	Does a manual change trigger an email alert within 15 minutes?
Data Traceability	Is the dashboard data queryable via Athena for long-term audit reports?

Architect's Final Instruction: You have built a fortress. Now, make sure you can see the walls. This dashboard is the final piece of the **Landed App** infrastructure puzzle.

WEEK 13

State Management, Scale Strategy & Resilience Planning

WEEKLY TECHNICAL SCOPE

1. Foundation: Orchestrating State for 100+ Account Estates.

2. Teaching Notes: Workspace vs. Per-Account State Backends.

3. Hands-on Lab: Migrating to a High-Performance S3/DynamoDB Backend.

4. Weekly Project: The "State-of-the-Union" Auditor.

Enterprise Extension: Operational Runbooks & Incident Response
Beyond detecting drift, operationalize how teams respond. Create incident response runbooks for different drift scenarios (manual console change, unauthorized policy, missing resource). Document post-incident reviews and process improvements to prevent recurrence.

Enterprise Extension: Disaster Recovery & Resilience Planning
Design recovery objectives (RTO/RPO) for your landing zone architecture. Document backup strategies, failover procedures, regional resilience patterns, and how to reconstruct a landing zone in a disaster scenario using IaC.

As the **Hexadigitall** environment expands, the "Blast Radius" of a corrupted state file becomes catastrophic. This week, we transition from simple automation to **Enterprise Scaling**. We focus on partitioning Terraform state to improve performance, utilizing Terragrunt for DRY

(Don't Repeat Yourself) configurations, and establishing the concurrency controls necessary for a global team to manage hundreds of accounts without collision.

"Scale is not just about having more; it is about how you handle the complexity of 'more'. In a 100-account estate, your state management is your heartbeat."

Chapter 13: Orchestrating State at Scale

As **Hexadigitall Technologies** scales beyond 100 accounts, the "Blast Radius" of infrastructure changes becomes the primary concern. Scaling strategies focus on **State Isolation**: the practice of breaking down a massive infrastructure state into smaller, decoupled units that can be updated independently.

The Scaling Paradox: Speed vs. Safety

To manage 100+ accounts efficiently, we implement a **Layered State Architecture**:

S1. **Core Layer:** Contains the Organization's root structure and TGW Hub. This state is extremely stable and rarely changes.

S2. **Governance Layer:** Manages SCPs, Config Aggregators, and IAM Identity Center. Changes here affect all accounts.

S3. **Account Layer:** A unique state file for every single account vended. If *Account 104* has a state corruption, *Account 105* remains fully operational.

Pre-Class Exercises: Designing for Density

Ex 1. **S3 Backend Stress Test:** Research the maximum object size for S3 and compare it to a typical terraform.tfstate for a VPC with 50 subnets.

Ex 2. **Dependency Mapping:** Draw a graph showing how an 'Account' state file references the 'Network Hub' state file using terraform_remote_state.

Ex 3. **Concurrency Audit:** Define a DynamoDB Locking strategy that prevents two automated AFT pipelines from modifying the same account simultaneously.

Architect's Warning: *At this scale, "Terraform Plan" times become a performance metric. If your plan takes 10 minutes to run, your CI/CD pipeline will fail under the weight of 100 accounts. Partitioning is the only solution.*

Workspaces vs. Per-Account Backends

The core challenge of managing 100+ accounts is **State Collision**. If multiple automated pipelines attempt to update different accounts that share a single state file or backend, the "Locking" mechanism will become a bottleneck.

1. The Workspace Approach (Centralized)

Workspaces allow you to use one set of configuration files to manage multiple distinct sets of infrastructure resources.

- **Pros:** Dry code; one single terraform init for all accounts.
- **Cons:** High risk. A "State rm" or "State push" error can accidentally target the wrong workspace. Performance degrades as the number of workspaces grows.

2. Isolated Backends (The Hexa-Standard)

In this model, every account (or small group of accounts) has its own S3 Key and DynamoDB Lock.

- **Pros:** Maximum blast-radius protection. Parallel execution across 100 accounts is physically isolated.
- **Cons:** Requires more complex automation (e.g., AFT or Terragrunt) to generate backend configurations dynamically.

The Architect's Verdict:

For **Hexadigitall**, we utilize **Isolated Backends**. While Workspaces are excellent for Dev/Stage/Prod within a single application, they lack the administrative separation required for a 100-account corporate Landing Zone.

Lab 13.1: Building the Isolated State Vault

We will provision the backend infrastructure in the **Management** or **Security** account to store the state of our 100+ spoke accounts.

Task: Defining the Backend Resources

```
# DynamoDB for State Locking
resource "aws_dynamodb_table" "state_lock" {
  name         = "hex-spoke-101-lock"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }
}

# S3 for State Storage
resource "aws_s3_bucket" "state_storage" {
  bucket = "hex-spoke-101-state-vault"

  lifecycle {
    prevent_destroy = true
  }
}
```

Success Criteria

[ ] S3 Bucket must have **Versioning Enabled** to allow for state recovery.

- [ ] Public Access Block must be applied to the S3 bucket to prevent state exposure.

Lab 13.2: The Migration Command

Transitioning state is a critical skill. You must ensure that no resources are duplicated or destroyed during the move.

Task: Initializing the Migration

1. Update your backend.tf with the new S3 and DynamoDB details from Lab 13.1.

```
terraform {
  backend "s3" {
    bucket         = "hex-spoke-101-state-vault"
    key            = "infrastructure/v1.tfstate"
    region         = "us-east-1"
    dynamodb_table = "hex-spoke-101-lock"
    encrypt        = true
  }
}
```

2. Execute the migration in your terminal:

```
$ terraform init -migrate-state
```

Verification Checklist

- [ ] Did Terraform prompt you to "copy the existing state to the new backend"? (Answer 'Yes').
- [ ] Check the S3 bucket: Is the v1.tfstate file present?
- [ ] Run terraform plan. Does it show "No changes"? (This confirms a successful migration).

> **Architect's Tip:** In a 100-account environment, you would never do this manually. You would use **Terragrunt** or **AFT** to generate these backend.tf blocks dynamically as part of the account vending process.

Project 13: The "State-of-the-Union" Auditor

The objective of this project is to create a **State Sentinel**. You will build a system that alerts the platform team if any state file in the 100-account estate is accessed manually (outside of CI/CD), if a lock is held for more than 4 hours (indicating a hung pipeline), or if versioning is disabled.

Project Deliverables

1. The Access Sentinel

Implement **S3 Bucket Notifications** on your State Vault. Trigger a Lambda function whenever a .tfstate file is read. If the IAM Principal is not the 'AFT-Execution-Role', fire a High-Severity alert to the Security Ops channel.

2. The Stale Lock Detector

Create a Python script (run via EventBridge/Lambda) that scans the DynamoDB State Lock table. Any item with a Created timestamp older than 4 hours must be flagged for manual inspection to prevent pipeline deadlocks.

3. The Integrity Report

Generate a STATE_HEALTH.md report that lists all 100+ state keys and confirms that **MFA-Delete** and **Object-Lock** are active on each, ensuring that not even a compromised admin can wipe the environment history.

Success Criteria

Detection Latency	Does a manual 'S3 Get' on a state file trigger an alert within 60 seconds?
Pipeline Recovery	Can the auditor identify a "Zombie" lock without disrupting active runs?
Immutability Proof	Can you provide a version history for a specific Spoke's state across three deployments?

Architect's Final Note: In the world of Landing Zones, your state is your memory. If you lose your memory, you lose your environment. Treat your .tfstate files with more reverence than your production databases.

WEEK 14

Capstone Delivery, Executive Handover & Architecture Defense

WEEKLY TECHNICAL SCOPE

1. **Foundation:** The Pillars of Operational Readiness.
2. **Teaching Notes:** Documentation as Infrastructure (DaI).
3. **Hands-on Lab:** Generating Automated Architecture Diagrams.
4. **Capstone Project:** The Full Hexadigitall Environment Sign-off.

The building is built, the security is armed, and the automation is humming. Now comes the most critical part: **The Handover**. This week is about professionalizing the Landing Zone. We focus on Runbooks, Disaster Recovery validation, and Cost Optimization reporting. You will prove that the environment is not just "ready to use," but **ready to be managed** by a Tier-1 Operations team without your constant intervention.

"A true specialist is measured by how well their system runs when they are no longer in the room. Documentation is the bridge between a project and a legacy."

Chapter 14: The Pillars of Operational Readiness

A Landing Zone is not "finished" until the operations team can maintain it without the original architect. At **Hexadigitall Technologies**, we measure readiness through four pillars that transform a technical project into a stable business asset.

The 4 Pillars of the Handover

To achieve "Operational Excellence," every specialist must deliver the following:

R1. **Incident Response Playbooks:** Clear, step-by-step instructions for common issues (e.g., "What to do if a Transit Gateway attachment fails").

R2. **Configuration Baselines:** A "Known-Good" snapshot of all 100+ account settings, enabling the team to identify drift instantly.

R3. **Financial Guardrails:** Automated reports that link cloud costs back to specific business units (Cost Centers) defined in Phase I.

R4. **Security Ownership:** A formal "Transfer of Keys" where root access is rotated and MFA is handed over to the permanent Security Operations Center (SOC).

Pre-Class Exercises: The Checklist

Ex 1. **The Break-Glass Audit:** Document the exact physical or digital location of the Root MFA device for the Management Account. Verify who has access.

Ex 2. **SLA Mapping:** Define the "Uptime Expectation" for the Network Hub. If the Hub is down, how many business-critical applications are affected?

Ex 3. **Contact Matrix:** Create ESCALATION_PATH.md. Who is called when the AFT pipeline fails at 3:00 AM?

Specialist Insight: *The mark of a master architect is the silence that follows their departure. If you are still receiving "emergency" calls three months after the handover, the system was not truly ready.*

Documentation as Infrastructure (Dal)

The greatest threat to a 100-account Landing Zone is **Tribal Knowledge**. If the logic of the network or the purpose of a specific SCP exists only in the architect's mind, the system is fragile. Dal ensures the "Why" is versioned alongside the "What."

1. The Self-Documenting Codebase

We utilize tools like terraform-docs to automatically generate Markdown files from our HCL variables and outputs.

- **Standard:** Every module must contain a README.md that is auto-updated by the CI/CD pipeline on every merge.
- **Logic:** Comments in code must explain *why* a decision was made (e.g., "Denying S3:PutBucketPolicy to prevent manual permission bypass") rather than just what the code does.

2. The Architecture Decision Record (ADR)

An ADR is a short text file that captures a significant architectural decision.

- **Format:** Title, Date, Status (Proposed/Accepted), Context, Decision, and Consequences.
- **Value:** When the team changes in two years, the ADR explains why **Hexadigitall** chose Transit Gateway over VPC Peering.

The Architect's Mantra:

If it isn't documented in the repo, it doesn't exist. If the documentation doesn't match the code, both are wrong.

Lab 14.1: Mapping the 100-Account Hub

A visual map is the first thing an operations team needs during an incident. We will automate this using Python.

Task: Scripting the Visual

```
from diagrams import Cluster, Diagram
from diagrams.aws.network import TransitGateway, VPC
from diagrams.aws.general import Users

with Diagram("Hexadigitall Global Hub", show=False):
    users = Users("Dev Teams")

    with Cluster("Network Hub Account"):
        tgw = TransitGateway("Core-TGW")

    with Cluster("Spoke Accounts (1-100)"):
        spokes = [VPC(f"Spoke-{i}") for i in range(1, 4)] # Simplified for viz

    users >> spokes >> tgw
```

Operational Steps

- [ ] Install the requirements: pip install diagrams.
- [ ] Run the script to generate hexadigitall_global_hub.png.
- [ ] Commit the image and the script to the /docs/arch-diagrams folder in your Git repo.

Lab 14.2: The Compliance Audit

We will automate the "Final Check" before sign-off.

Task: Running the Cross-Account Tag Scan

Use the AWS CLI with the --resource-type-filters flag to identify any resource missing the Environment tag.

```
$ aws resourcegroupstaggingapi get-resources \ --tag-filters
Key=Environment,Values= \ --region us-east-1
```

Final Validation

- [ ] Ensure zero resources are returned (indicating 100% tag compliance).
- [ ] Export the results to a HANDOVER_AUDIT_MARCH_2026.csv.

Architect's Achievement: You have moved from a builder to a governor. This report is your technical "Proof of Work" that the environment is professional, compliant, and ready for business.

Capstone: The Hexadigitall Master Landing Zone

The Capstone Project requires the deployment and formal documentation of a production-ready, multi-account environment. You must prove that **Phase I through IV** are integrated into a single, cohesive GitOps machine.

Final Deliverables (The Handover Pack)

1. The "Golden" Git Repository

A clean, modularized Terraform/AFT repository containing the Global Baseline, Network Hub, and Security Spoke configurations. Every module must have 100% documentation coverage.

2. The Operational Runbook

A comprehensive RUNBOOK.md defining how to: vend a new account, remediate a drift alert, rotate the TGW encryption keys, and decommission a project safely.

3. The Compliance Evidence Locker

A collection of automated reports (Config Aggregator, Security Hub, and Tag Audits) proving that the 100-account estate satisfies all corporate guardrails.

Final Certification Rubric

Automation	Can a new compliant account be vended with zero manual console interaction?
Resilience	Is the state management partitioned to prevent a single point of failure?
Clarity	Could a new DevOps engineer understand the architecture in under 30 minutes?

End of Curriculum

You have demonstrated the technical mastery and operational discipline required of a **Hexadigitall Specialist**. The cloud is no longer a wilderness; it is a governed engine.

Phase IV Knowledge Assessment

📋 Assessment Overview

Questions: 20 multiple-choice questions
Time Estimate: 60-80 minutes
Passing Score: 14/20 (70%)
Coverage: Weeks 12-14 (Compliance, Lifecycle, Capstone Delivery)

1. What is the primary purpose of AWS Config Rules in a Landing Zone?

A) To monitor resource costs
B) To enforce compliance and detect configuration drift
C) To manage user permissions
D) To create backups

2. Which AWS service is best suited for automated remediation of non-compliant resources?

A) AWS Config
B) AWS Systems Manager Automation
C) AWS CloudWatch
D) AWS Trusted Advisor

3. What is "Configuration Drift" in the context of infrastructure management?

A) Network latency issues
B) Manual changes that deviate from the desired state
C) Automatic scaling events
D) Resource migration between regions

4. Which service provides immutable log storage in a Landing Zone?

A) AWS CloudWatch Logs
B) AWS S3 with Object Lock
C) AWS RDS
D) AWS DynamoDB

5. What is the role of "Account Baselines" in Landing Zone management?

A) To track resource usage
B) To define the approved configuration state for accounts
C) To manage billing
D) To create user accounts

6. Which AWS service enables cross-account log aggregation?

A) AWS Organizations
B) AWS CloudTrail Organization Trails
C) AWS Config Aggregators
D) All of the above

7. What is the primary benefit of "Documentation as Infrastructure" (DaI)?

A) Reduces development time
B) Ensures operational knowledge transfer
C) Lowers infrastructure costs
D) Improves code quality

8. Which service provides compliance evidence for regulatory frameworks?

A) AWS Artifact
B) AWS Config
C) AWS Trusted Advisor
D) AWS Cost Explorer

9. What is the purpose of "Operational Runbooks" in a Landing Zone?

A) To document code changes
B) To provide step-by-step incident response procedures
C) To track project timelines
D) To manage budgets

10. Which AWS service enables automated cost optimization?

A) AWS Cost Explorer
B) AWS Budgets
C) AWS Compute Optimizer
D) All of the above

11. What is "Account Decommissioning" in Landing Zone lifecycle management?

A) Deleting unused accounts
B) The secure process of retiring accounts while preserving compliance
C) Upgrading account tiers
D) Migrating accounts between regions

12. Which service provides continuous security assessment?

A) Amazon GuardDuty
B) AWS Security Hub
C) Amazon Inspector
D) All of the above

13. What is the benefit of "Immutable Infrastructure"?

A) Reduces costs
B) Prevents configuration drift and ensures consistency
C) Improves performance
D) Simplifies development

14. Which AWS service provides automated backup management?

A) AWS Backup
B) AWS Storage Gateway

C) Amazon S3
D) AWS Snowball

15. What is "Disaster Recovery Testing" in Landing Zone operations?

A) Testing backup restoration
B) Validating failover procedures and recovery time objectives
C) Checking network connectivity
D) Monitoring resource usage

16. Which service enables automated compliance reporting?

A) AWS Config
B) AWS Security Hub
C) AWS Audit Manager
D) All of the above

17. What is the purpose of "Change Management" in Landing Zone operations?

A) To track code changes
B) To ensure changes are approved, tested, and documented
C) To manage user permissions
D) To monitor performance

18. Which AWS service provides centralized logging for multiple accounts?

A) AWS CloudWatch
B) AWS CloudTrail
C) Amazon S3
D) AWS Kinesis

19. What is "Operational Readiness" in the context of Landing Zone handover?

A) Having all features implemented

B) Ensuring operations teams can maintain the environment independently

C) Meeting project deadlines

D) Achieving cost targets

20. Which service enables automated architecture diagram generation?

A) AWS Systems Manager

B) AWS Config

C) AWS CloudFormation

D) Amazon Q

🎯 **Phase IV Completion**

Congratulations on completing Phase IV! Your Landing Zone is now operationally ready.

Proceed to the final comprehensive certification exam to validate your complete mastery of Landing Zone architecture.

Landing Zone Specialist Certification

🎯 **Final Assessment Overview**

Questions: 25 multiple-choice questions

Time Estimate: 75-90 minutes

Passing Score: 19/25 (76%)

Coverage: All 4 phases + comprehensive integration

1. Which AWS service provides the foundational structure for multi-account governance in a Landing Zone?

A) AWS Control Tower
B) AWS Organizations
C) AWS Resource Access Manager
D) AWS Systems Manager

2. What is the primary purpose of AWS Control Tower Account Factory for Terraform (AFT)?

A) To manage IAM users
B) To automate account provisioning and customization
C) To monitor resource usage
D) To create network connectivity

3. Which service enables self-service infrastructure deployment with governance?

A) AWS Control Tower
B) AWS Service Catalog
C) AWS Organizations
D) AWS Resource Groups

4. What is "Configuration Drift" and how is it detected?

A) Network latency between regions
B) Manual changes outside IaC, detected by AWS Config
C) Automatic scaling events
D) Resource migration between accounts

5. Which AWS service provides immutable log storage for compliance?

A) AWS CloudWatch Logs
B) Amazon S3 with Object Lock
C) AWS RDS
D) AWS DynamoDB

6. What is the role of "Launch Constraints" in AWS Service Catalog?

A) To limit product launches
B) To specify the IAM role for resource creation
C) To define cost limits
D) To restrict regions

7. Which service enables automated remediation of non-compliant resources?

A) AWS Config
B) AWS Systems Manager Automation
C) AWS CloudWatch
D) AWS Trusted Advisor

8. What is "GitOps" in Landing Zone management?

A) Using Git for version control only
B) Treating Git as the source of truth for infrastructure
C) Managing permissions through Git
D) Storing logs in Git

9. Which AWS service provides compliance evidence for regulatory frameworks?

A) AWS Artifact
B) AWS Config
C) AWS Trusted Advisor
D) AWS Cost Explorer

10. What is the benefit of "Immutable Infrastructure"?

A) Reduces costs
B) Prevents configuration drift and ensures consistency
C) Improves performance
D) Simplifies development

11. Which service enables cross-account log aggregation?

A) AWS Organizations
B) AWS CloudTrail Organization Trails
C) AWS Config Aggregators
D) All of the above

12. What is "Operational Readiness" in Landing Zone handover?

A) Having all features implemented
B) Ensuring operations teams can maintain the environment independently
C) Meeting project deadlines
D) Achieving cost targets

13. Which AWS service provides automated cost optimization?

A) AWS Cost Explorer
B) AWS Budgets
C) AWS Compute Optimizer
D) All of the above

14. What is the purpose of "Account Baselines"?

A) To track resource usage
B) To define approved configuration states

C) To manage billing
D) To create user accounts

15. Which service provides continuous security assessment?

A) Amazon GuardDuty
B) AWS Security Hub
C) Amazon Inspector
D) All of the above

16. What is "Documentation as Infrastructure" (DaI)?

A) Storing docs in infrastructure
B) Maintaining documentation alongside code
C) Using docs to build infrastructure
D) Infrastructure that generates docs

17. Which AWS service enables automated backup management?

A) AWS Backup
B) AWS Storage Gateway
C) Amazon S3
D) AWS Snowball

18. What is "Change Management" in Landing Zone operations?

A) Tracking code changes
B) Ensuring approved, tested, documented modifications
C) Managing user permissions
D) Monitoring performance

19. Which service enables automated compliance reporting?

A) AWS Config
B) AWS Security Hub
C) AWS Audit Manager
D) All of the above

20. What is "Account Decommissioning"?

A) Deleting unused accounts
B) Secure retirement while preserving compliance
C) Upgrading account tiers
D) Migrating accounts

21. Which AWS service provides centralized logging for multiple accounts?

A) AWS CloudWatch
B) AWS CloudTrail
C) Amazon S3
D) AWS Kinesis

22. What is the role of "State Management" in Terraform?

A) Managing user sessions
B) Tracking infrastructure current state
C) Controlling access
D) Monitoring performance

23. Which service enables automated architecture diagram generation?

A) AWS Systems Manager
B) AWS Config
C) AWS CloudFormation
D) Amazon Q

24. What is "Disaster Recovery Testing"?

A) Testing backup restoration
B) Validating failover procedures and RTOs
C) Checking network connectivity
D) Monitoring resource usage

25. Which principle ensures Landing Zone maintainability?

A) Maximum automation

B) Infrastructure as Code

C) Continuous monitoring

D) All of the above

🎓 Certification Complete

Congratulations! You have successfully completed the Hexadigitall Landing Zone Specialist certification.

Your comprehensive understanding of Landing Zone architecture, from foundational governance through operational excellence, positions you as a trusted cloud infrastructure specialist.

Final Certification

Technical Terms & Definitions

Governance & Structure

AWS Organizations: The foundational service for account consolidation. It enables hierarchical grouping into OUs and centralizes billing/policy enforcement.

Control Tower: An AWS managed service that orchestrates Organizations, SSO, and Config to set up and govern a secure multi-account environment.

OU (Organizational Unit): A logical container for accounts within an organization. Policies (SCPs) applied to an OU are inherited by all child accounts.

SCP (Service Control Policy): A JSON policy that sets the maximum available permissions for an account or OU. It acts as a "filter" and cannot grant permissions, only restrict them.

Blast Radius: The total impact surface area of a technical failure or security breach. Landing Zones use account boundaries to minimize this area.

Automation & State

AFT (Account Factory for Terraform): A GitOps-driven engine that automates account creation and the application of custom Terraform baselines.

Drift: When the "Actual State" of a cloud resource deviates from the "Desired State" defined in HCL code, usually due to manual console changes.

Idempotency: The property where an automation script (like Terraform) produces the exact same result whether run once or one thousand times.

State Locking: A mechanism using DynamoDB to prevent multiple concurrent Terraform executions from writing to the same .tfstate file and causing corruption.

Terragrunt: A thin wrapper for Terraform that provides extra tools for keeping configurations DRY (Don't Repeat Yourself) and managing multiple remote states.

Networking & Security

Transit Gateway (TGW): A regional network hub that acts as a cloud router, simplifying VPC peering by connecting thousands of VPCs and on-premises networks.

RAM (Resource Access Manager): A service used to share AWS resources (like TGWs or Subnets) across accounts within an organization.

PrivateLink: A technology that provides private connectivity between VPCs and AWS services without using public IP addresses or traversing the internet.

Guardrails: High-level rules. *Preventative* guardrails (SCPs) stop actions; *Detective* guardrails (AWS Config) notify you when a violation has already occurred.

IAM Identity Center (SSO): A centralized service to manage single sign-on access to all AWS accounts and cloud applications from a single Identity Provider.

The "Day 2" Recovery Matrix

Domain / Issue	Probable Root Cause	Standard Remediation
STATE: Lock Error	A CI/CD runner crashed or timed out before releasing the DynamoDB lock.	Locate LockID in error output. Verify no active process. Run: terraform force-unlock [ID].
AFT: Provisioning Hang	Service Quota limit reached in target region (e.g., max VPCs) or Control Tower hang.	Check Step Function "Failure" status in AFT-Management. Request Quota increase via Service Quotas console.
IAM: "Entity Already Exists"	Manual console changes created a role/policy that Terraform is now trying to create.	Manually delete the console resource or use terraform import to bring it under code control.
SCP: Access Denied	Conflict between Local IAM "Allow" and a restrictive "Deny" at the OU level.	Use IAM Policy Simulator. Select "Simulation with SCPs". Identify the specific blocking policy ID.
TGW: No Connectivity	Missing TGW Attachment, or Route Table propagation is not enabled for the spoke.	Verify Spoke VPC Route Table points 0.0.0.0/0 to TGW. Verify TGW Route Table has an active propagation from the spoke.
DNS: Resolution Failure	Route 53 Resolver Rules not shared via RAM or VPC not associated with the Hub zone.	Check RAM shares in Spoke. Verify "VPC Association" in the Route 53 Private Hosted Zone console.
DRIFT: Config Non-Compliant	Remediation failed due to insufficient SSM permissions or resource deletion.	Check SSM Automation execution logs. Ensure the Config-Remediation-Role has appropriate PUT permissions.

Emergency Protocol:

If an issue cannot be resolved via code, utilize the **Break-Glass Role** (managed in IAM Identity Center). Document all manual actions in the POST_MORTEM.md repo and trigger a terraform apply immediately after to restore the baseline.

HEXADIGITALL TECHNOLOGIES

Certification of Completion

This document certifies that the bearer has completed the 14-week **Architecting Landing Zones** specialist roadmap, demonstrating proficiency in multi-account governance, automated security remediation, and enterprise state scaling.

DIRECTOR OF CLOUD GOVERNANCE

DATE OF ISSUE: MARCH 2026

"You have built more than a cloud; you have built a legacy of operational excellence. Now, go and govern the future."

FINAL PAGE | END OF ROADMAP

HEXADIGITALL

Project Credits & Contributions

Curriculum Authority:

Hexadigitall Technologies — Cloud Governance Division

Lead Architect:

The Specialist Engineering Team

Technical Standards:

AWS Well-Architected Framework (Multi-Account Pillar)

Azure Cloud Adoption Framework (CAF)

Production Environment:

The Hexadigitall Landing Zone v4.2 (2026 Edition)

All code snippets, architectural diagrams, and governance patterns contained within this roadmap are the intellectual property of **Hexadigitall Technologies**. Unauthorized distribution or replication of the "Landed" application logic is strictly prohibited.

© 2026 HEXADIGITALL TECHNOLOGIES. ALL RIGHTS RESERVED.

HEXA-REF: 2026-LZ-SPEC-01